



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

深度学习训练系统的 GPU 内存优化研究

作者姓名: 杨永新

指导教师: 冯晓兵 研究员

中国科学院计算技术研究所

学位类别: 工程硕士

学科专业: 计算机技术

培养单位: 中国科学院计算技术研究所

2021 年 6 月



中国科学院大学

University of Chinese Academy of Sciences

硕士学位论文

深度学习训练系统的 GPU 内存优化研究

作者姓名: 杨永新

指导教师: 冯晓兵 研究员

中国科学院计算技术研究所

学位类别: 工程硕士

学科专业: 计算机技术

培养单位: 中国科学院计算技术研究所

2021 年 6 月

Research on GPU memory optimization for deep learning
training system

A Thesis Submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Science
in Computer Technology

by

Yang Yongxin

Supervisor: Professor Feng Xiaobin

Institute of Computing Technology
Chinese Academy of Sciences

June, 2021

中国科学院大学

研究生学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：杨永新
日期：2021年5月31日

中国科学院大学

学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密的学位论文在解密后适用本声明。

作者签名：杨永新
日期：2021年5月31日

导师签名：王明
日期：2021.5.31

摘 要

深度学习应用已经普遍应用于日常生活,在 GPU 上训练深度学习模型也已成为主流的选择。但是,现代 GPU 的内存容量增长趋缓,在 GPU 上训练深度模型越来越受到内存不足的限制,内存的局限性表现得愈发突出。

深度学习训练过程中存在两方面的内存使用低效: 1)梯度反向传播的训练方法使得数据的复用特征与其在网络中的位置密切相关,现有的深度学习计算系统尚未充分地利用这些特征,造成部分数据长期驻留在 GPU 上; 2)现有深度学习训练系统的通用内存分配器,由于无法充分地利用 DNN 训练的全局信息,容易出现内存碎片。

针对上述两个问题,本文提出了内存高效的 DNN 训练系统 METS,核心在于通过自动内存交换降低内存负载峰值和全局内存分配优化减少内存碎片。本文的主要贡献有:

1) 提出了一种 Profile 制导的基于代价模型的自动内存交换策略。设计了一种新颖的 Profile 模式以获取 DNN 训练负载的访存模式,并提出了动态带权缺失时长代价模型用于交换内存块选择和交换事件调度,整个过程不需要感知计算图结构,可以灵活应对 DNN 应用的快速变化。

2) 提出了一种基于全局访存模式的内存分配算法。利用 Profile 中内存块生存期的重叠关系构建冲突图,并将问题抽象为二维条带包装问题进行优化。开发了一种简单的启发式方法,以快速生成良好的近似解。通过全局内存分配优化,实现数据之间更高级的共享内存,进一步减少内存碎片。

本文基于 TensorFlow 实现了一个原型系统 METS。实验结果表明,系统可以有效地降低 GPU 的内存占用,在不增加性能开销的情况下(性能开销<5%),内存占用最高降低 32%。同时,相比于目前工业界领先的训练系统 TensorFlow, METS 可以支持更大的 batchsize 正确运行,最高可达 2.1 倍,从而允许算法研究人员探索更多的网络结构设计空间。

关键词: 深度学习, 内存交换, 内存复用, 代价模型, 全局分配优化

Abstract

Deep learning applications have been widely used in daily life, and training deep learning models on GPUs has also become a mainstream choice. However, the memory capacity of modern GPUs has slowed down, and training deep models on GPUs is increasingly restricted by insufficient memory, and the limitations of memory have become more prominent.

There are two aspects of memory inefficiency in the deep learning training process: 1) The gradient backpropagation training method makes the data reuse characteristics closely related to its position in the network, and the existing deep learning computing systems have not yet fully utilized these Characteristic, causing some data to reside on the GPU for a long time; 2) The general memory allocator of the current deep learning training system is prone to memory fragmentation due to the inability to make full use of the global information of DNN training.

In response to the above two problems, this paper proposes a memory-efficient DNN training system METS. The core is to reduce the peak memory load through automatic memory swap and optimize the global memory allocation to reduce memory fragmentation. The main contributions of this article are:

1) A Profile-guided automatic memory swap strategy based on cost model is proposed. A novel Profile mode is designed to obtain the memory access mode of DNN training load, and a dynamic weighted absence time cost model is proposed for swapping block selection and swapping event scheduling. The whole process does not need to know the graph structure, which can be flexible to deal with the rapid changes in DNN applications.

2) A memory allocation algorithm based on global memory access mode is proposed. The conflict graph is constructed by using the overlap relationship of the memory block lifetime in the Profile, and the problem is abstracted as a two-dimensional strip packaging problem for optimization. And developed a simple heuristic method to quickly generate a good approximate solution. Through global memory allocation optimization, a higher level of shared memory between data is realized, and memory fragmentation is further reduced.

This article implements a prototype system METS based on TensorFlow. Experimental results show that the system can effectively reduce the memory footprint of the GPU, and the memory footprint can be reduced by up to 32% without increasing performance overhead (performance overhead <5%). At the same time, compared to

the current industry-leading training system TensorFlow, METS can support larger batchsize to run correctly, up to 2.1 times, allowing algorithm researchers to explore more network structure design space.

Keywords: deep learning, memory swap, memory reuse, cost model, global allocation optimization

目 录

第 1 章 引言	1
1.1 本文的研究背景和动机	1
1.1.1 研究背景	1
1.1.2 深度学习训练过程	3
1.1.3 深度学习训练系统内存行为	6
1.1.4 内存使用低效问题定义	10
1.2 本文的主要贡献	11
1.3 论文的组织结构	12
第 2 章 相关研究	15
2.1 深度学习框架内存解决方案	15
2.1.1 异构系统简介	15
2.1.2 动态内存池简介	15
2.1.3 深度学习框架内存减少技术	16
2.2 深度学习训练内存优化的研究现状	17
2.2.1 模型压缩	17
2.2.2 内存共享	18
2.2.3 内存交换	18
2.2.4 输出重计算	19
2.3 本章小结	20
第 3 章 内存高效训练系统总览	21
3.1 METS 系统结构	21
3.2 METS 系统工作流程	23
3.3 METS 系统模块介绍	24
3.3.1 内存行为分析模块	24
3.3.2 自动内存交换模块	27
3.3.3 全局内存分配优化模块	27
3.4 METS 的局限性	28

第 4 章 基于代价模型的自动内存交换技术	29
4.1 DNN 内存负载行为分析	29
4.2 主机和 GPU 内存交换行为分析	31
4.2.1 主机和 GPU 内存交换开销刻画	32
4.2.2 代价模型设计原则分析	33
4.3 交换内存块代价模型介绍	35
4.4 交换事件调度设计	38
4.5 工程实现介绍	39
4.6 内存交换策略对比分析	40
4.7 本章总结	41
第 5 章 基于全局访存行为的内存分配优化技术	43
5.1 动态内存池碎片问题剖析	43
5.2 内存分配优化问题抽象	45
5.3 MILP 求解方法介绍	47
5.4 Best-Fit 启发式算法求解介绍	49
5.5 内存分配优化模块工程实现	51
5.5.1 全局访存行为模式提取	51
5.5.2 内存分配执行流程	51
5.5.3 单 GPU 不可训练模型解决方案	52
5.6 动态卷积工作空间分配优化	52
5.6.1 卷积工作空间介绍	52
5.6.2 卷积工作空间动态分配机制	53
5.7 本章小结	54
第 6 章 实验评估	57
6.1 实验平台	57
6.2 实验设计原则	57
6.2.1 Benchmark 选择	57
6.2.2 实验设计	59
6.3 METS 总体评测	60
6.3.1 训练速度对比	60

6.3.2 内存占用对比	62
6.4 基于代价模型的自动内存交换实验评测	63
6.4.1 自动内存交换模块的内存负载和内存占用收益	64
6.4.2 一次迭代内存负载变化分析	65
6.4.3 一次迭代内存占用变化分析	66
6.5 基于全局访存行为的内存分配优化实验评测	67
6.5.1 内存分配优化算法内存降低	67
6.5.2 内存分配优化算法训练耗时评估	69
6.5.3 启发式算法开销评估	70
6.5.4 动态卷积工作空间分配测评	71
6.5.5 内存碎片降低情况	72
6.6 本章小结	73
第 7 章 总结与展望	75
7.1 本文工作总结	75
7.2 下一步研究方向	77
参考文献	78
致 谢	83
作者简历及攻读学位期间发表的学术论文与研究成果	84

图目录

图 1.1 Tesla 系列 GPU 内存容量发展趋势	2
图 1.2 深度学习训练系统架构.....	3
图 1.3 前向传播和反向传播.....	4
图 1.4 卷积神经网络示例 (LeNet)	5
图 1.5 不同 DNN 训练内存占用分解.....	7
图 1.6 ResNet18 10 次迭代内存负载曲线	8
图 1.7 ResNet 前 10 次迭代甘特图	10
图 3.1 METS 系统结构示意图	22
图 3.2 METS 工作流程	23
图 3.3 Profile 数据格式	25
图 4.1 DNN 训练负载 GPU 数据驻留示意图	30
图 4.2 DNN 内存负载变化曲线.....	31
图 4.3 内存块访问间隔累积分布函数	33
图 4.4 访问间隔(a)和内存块大小(b)的箱线图.....	34
图 4.5 ResNet 训练中的访问间隔和相应的内存块大小.....	34
图 4.6 缺失时长示意图.....	36
图 4.7 工程实现抽象.....	39
图 4.8 基于代价函数的内存交换算法流程	40
图 5.1 内存碎片产生示意图.....	45
图 5.2 二维条带打包问题示意图	46
图 5.3 Best-Fit 启发式算法示意图	49
图 5.4 常见 CNN 各层执行时间分解.....	53
图 5.5 卷积工作空间动态分配策略	54

图 6.1 METS 内存占用优化结果	60
图 6.2 开源解决方案训练速度对比	62
图 6.3 开源解决方案内存占用对比	63
图 6.4 自动内存交换模块内存负载收益	64
图 6.5 自动内存交换模块内存占用收益	65
图 6.6 ResNet18 训练一次迭代的内存负载变化曲线.....	66
图 6.7 ResNet18 一次训练迭代的内存占用曲线.....	66
图 6.8 全局内存分配优化内存占用降低	68
图 6.9 CNN 模型运行时开销降低情况.....	69
图 6.10 Seq2Seq 和 BERT 的运行时开销降低情况.....	70
图 6.11 动态卷积工作空间分配优化效果	71
图 6.12 全局内存分配优化前后甘特图对比	73
图 7.1 METS 优化效果总体示意	76

表目录

表 1.1 2018-2020 重大 AI 模型成果(OpenAI) 2

表 4.1 现有解决方案内存交换策略对比 41

表 6.1 实验选取 Benchmark..... 58

表 6.2 MILP 与 Best-Fit 启发式算法对比 70

表 6.3 Best-Fit 启发式算法执行耗时 71

第 1 章 引言

1.1 本文的研究背景和动机

1.1.1 研究背景

深度学习算法^[1]在各个领域中都取得了巨大的成功，例如图像分类，自然语言处理，目标检测，语音识别等。随着大数据的发展、易于获得的 GPGPU 以及神经网络建模技术的进步，在 GPU 上训练深度学习模型已成为一种流行的选择。但是，现代 GPU 的内存容量增长趋缓，在 GPU 上训练深度模型越来越受到内存不足的限制，内存的局限性表现得愈发突出。

深度学习模型是一个计算密集型过程，需要大量数据和大量计算能力。先前的研究表明，更深的 DNN 可以显著提高模型性能。尤其是，图形处理单元 (GPU) 的有效使用使得有可能在庞大的数据集上训练复杂的 DNN。算法设计人员为了获得更好的准确性，将网络设计的越来越深和越来越宽；越来越深体现在模型的层数不断地增长，越来越宽体现在模型的通道数和特征维度越来越大。例如，在图像识别中，ILSVRC 2012 的获奖者 AlexNet^[2]仅由九层组成，并具有顺序结构。GoogLeNet 引入了所谓的初始模块，这是扩展 DNN 的技术。ResNet 包含 50 多个层；最近的网络 Inception ResNet^[3]，通过 Resception 模块扩展了 ResNet，甚至比 ResNet 和 GoogLeNet 大。而神经机器翻译 (NMT)^[4]模型由 8 个层组成，使用注意力机制，并且 NMT 模型中的大多数层是连续展开的顺序层，带来不可忽视的内存消耗。这一趋势在最新的语言模型上表现得更为明显，如表 1.1 所示为 OpenAI 的技术报告数据摘录，从 2018 年到 2020 年两年内，GPT 的模型参数量从 110M 发展到了 175B，增长了 1500 多倍。

相比于深度学习模型规模的增长，GPU 内存容量发展趋于平缓，如图 1.1 所示为 Tesla 系列 GPU 内存容量的发展趋势，从 2013 年到 2020 年，GPU 内存容量仅仅从 12G 增加到 16G。GPU DRAM 的有限大小一直是研究人员探索更深，更广泛的 DNN 以获得更好的泛化性能的主要瓶颈。例如，由 16 个计算密集型卷积层组成的 VGG-16^[5]对于批量大小 256 要求总共 28GB 的内存使用。作为一种代

表性的非线性网络,Inception-V4^[3]要求高达 45GB 的内存,以使整个网络都在 GPU 上进行训练。但是,到目前为止,商用 NVIDIA Volta^[6]架构提供的最大 GPU 内存容量为 32GB。GPU 的内存不足限制了深度学习从业人员训练越来越广泛的深度学习模型。

表 1.1 2018-2020 重大 AI 模型成果(OpenAI)

Table 1.1 2018-2020 Major AI Model Achievements (OpenAI)

时间	机构	模型名称	模型规模	数据规模
2018.06	OpenAI	GPT	110M	4GB
2018.10	Google	BERT	330M	16GB
2019.02	OpenAI	GPT-2	1.5B	40GB
2019.07	Facebook	RoBERTa	3.3B	160GB
2019.10	Google	T5	11B	800GB
2020.06	OpenAI	GPT-3	175B	45TB

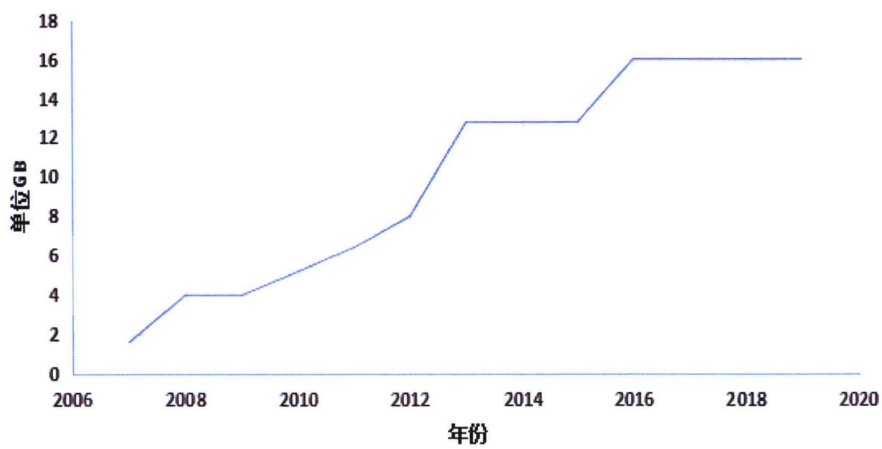


图 1.1 Tesla 系列 GPU 内存容量发展趋势

Figure 1.1 The development trend of Tesla series GPU memory capacity

内存容量不足给 DNN 训练带来的主要限制有两方面,首先,对于训练 DNN,我们只能使用较小的 BatchSize,以避免内存用尽的风险,因此收敛速度较慢。其次,使用如此大的 DNN 进行训练可能会在集群环境中需要更多的机器。最后,为了使模型可以被放入单个设备中进行训练,神经网络设计的灵活性受到了限制。在使用 GPU 和边缘设备时,与 CPU 相比,它们的内存存储量要小得多,并且可扩展性也要差得多,因此高内存消耗更加严重。

总而言之，有限的 GPU 设备内存容量和不断增加的模型复杂性之间的差距使得内存优化成为必要的要求。

1.1.2 深度学习训练过程

深度学习训练系统通常都是异构计算架构，训练的过程由正向传播和反向传播构成，一个模型从初始化到训练收敛通常需要大量的迭代优化，单次迭代由一次正向传播和一次反向传播构成。

深度学习训练系统使用一台具有主机 CPU，主机内存和异构加速器板的计算机，该加速器板是具有本地存储的 GPU，如图 1.2 所示。这里的加速器还可以是 FPGA 和各种 ASIC 芯片。在训练的迭代开始之前，参数已在 GPU 内部初始化。然后，将一组输入数据从磁盘加载到 GPU，并执行针对该组输入数据的训练计算并更新参数。训练完成后，输出参数将存储到磁盘。

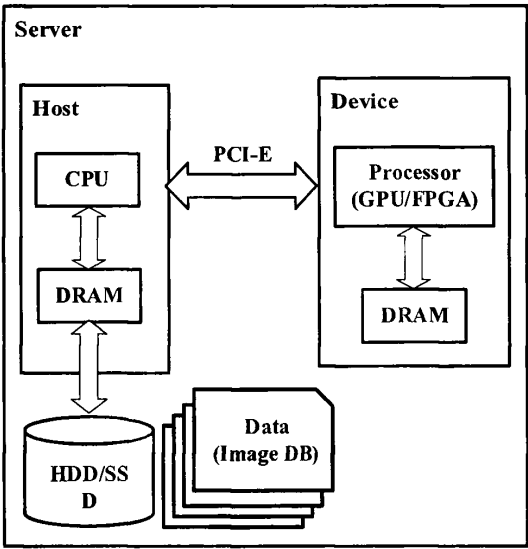


图 1.2 深度学习训练系统架构

Figure1.2 System architecture for DNN training

深度学习的训练过程主要基于大量的数据，这些数据由输入和对应的标签构成。如图 1.3 所示，训练过程分为两个阶段：前向计算和反向传播计算。具体来说，反向传播计算将计算损失函数，并搜索损失函数的梯度，该梯度可用于调整参数以提高准确性。在深度学习的训练中，主要包含四种类型的数据结构：特征

图, 权重, 梯度和工作区。特征图是在前向计算中产生的中间结果。梯度图是在反向传播计算中生成并由当前反向计算层消耗的中间结果。权重值决定输入将对输出产生多大影响。工作空间是层内存储空间, 用于加快层计算速度。这里要注意的是, 工作空间需要额外但临时的 GPU DRAM, 其目的是让卷积算法有更好性能。在前向计算中, 输入特征图通过网络拓扑结构在前向方向上馈送到当前图层。每个隐藏层都接受输入数据, 按照激活函数对其进行处理, 然后将其输出传递给后续层。

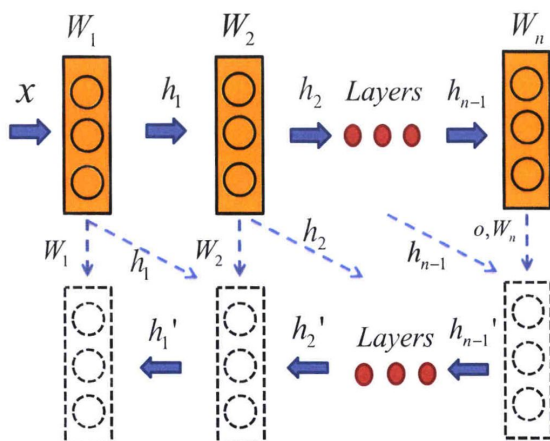


图 1.3 前向传播和反向传播

Figure1.3 Forward-propagation and backward-propagation

这里将 CNN 中 LeNet^[7]的训练作为深度学习网络训练的一个实例进行概述。网络的训练包括三个阶段: 正向传播, 反向传播和参数更新。正向传播通过将一组图像分类为给定数量的类别 (即执行推理阶段)。反向传播根据前向传播中获得的误差为每个层计算神经元数据和参数的梯度, 并将梯度传播到之前的层。参数更新阶段通过减去在反向传递中获得的梯度乘以学习率来更新参数。重复这三个阶段, 直到满足收敛标准为止。

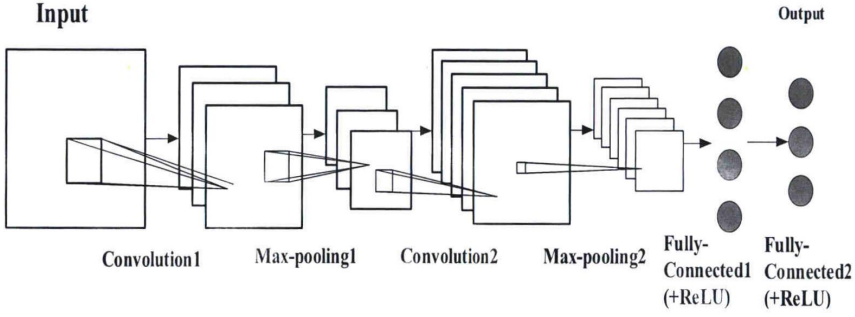


图 1.4 卷积神经网络示例 (LeNet)

Figure1.4 An example of Convolutional Neural Network(LeNet).

图 1.4 中描述的示例 CNN（称为 LeNet）由卷积层，MaxPooling 和全连接层组成。卷积层使用 $m \times m$ 尺寸的二维滤波器（即参数数据）在 $N \times N$ 个像素的神经元数据上计算卷积。输出数据通过非线性激活函数 $\sigma(x)$ ，例如 ReLU 或 Sigmoid。注意，对于某些层，可以省略激活功能（即，可以应用相同的功能）。卷积层的计算描述如下：

$$x_{i,j}^l = \sum_{b=0}^{m-1} \sum_{a=0}^{m-1} w_{a,b}^{l-1} y_{(i+a)(j+b)}^{l-1} + bias \quad (1.1)$$

$$y_{i,j}^l = \sigma(x_{i,j}^l) \quad (1.2)$$

其中 $x_{i,j}^l$ 是第 l 层 (i,j) 处的数据， $y_{i,j}^l$ 是第 l 层 (i, j) 处的激活（即将激活函数应用于 $x_{i,j}^l$ ）， $w_{a,b}^{l-1}$ 是 (a,b) 处的加权滤波器数据，并且 $bias$ 为所有像素添加一个常数值，以调整激活阈值。

池化层将神经元数据缩减为较小的数据像素。最常使用的池化是最大池化 (Max-pooling)，它为每个 $k \times k$ 区域提取最大数据。还有平均池化用于某些 DNN，例如 GoogLeNet [9]。Max-pooling 层的计算描述如下：

$$y_{i,j}^l = \max\{y_{i+a,j+b}^{l-1} \mid a, b \in [0, k-1]\} \quad (1.3)$$

全连接层计算神经元和参数之间所有组合的乘法。输出数据通过非线性激活函数 $\sigma(x)$ （例如 ReLU 或 S 形曲线）作为卷积层。全连接层的计算描述如下：

$$x_i^l = \sum_j w_{j,i}^{l-1} y_j^{l-1} + bias \quad (1.4)$$

$$y_i^l = \sigma(x_i^l) \quad (1.5)$$

其中 x_i 是第 l 层上的第 i 个数据, y_i 是第 l 层上的第 i 个激活, $x_{i,j}^{l-1}$ 是第 j 个输入和第 i 个输出之间的权重参数。最终的 Softmax 层将输出数据归一化为分类的概率分布。反向传播方法是一种迭代优化方法,它通过将神经元数据的误差梯度传播到较早的层来逐渐使误差最小化。

反向传播需要的内存是前向的两倍,因为反向传播除了需要用到前向传播的特征图和参数之外,还需要分配内存用于存放梯度图。

最后,使用所有层的参数梯度来更新参数。可以使用批处理方法来生成精确的解决方案,该方法使用整个输入数据进行一次迭代。由于批处理方法非常慢,因此使用 SGD^[8] (随机梯度下降) 方法。基本的 SGD 方法使用单个输入数据更新参数,而不是针对每次迭代随机地更新一批数据。已经提出了许多改进的优化方法来使用 SGD 更新参数,例如 Momentum^[10] 和 AdaGrad^[11]。mini-batch 方法 (一次批处理只需要少量输入数据 (通常为 10 到 1000)) 也可用于提高 SGD 的性能。mini-batch 方法的 SGD 使用以下公式更新参数:

$$w_{t+1} = w_t - \frac{\eta}{|M|} \sum_{m=1}^M \Delta W_t \quad (1.6)$$

其中 ΔW_t 是第 t 次迭代的参数梯度, η 是学习率, M 是最小批次大小。还要注意,随着最小批处理大小变大,特征图所需的内存大小也变大。

1.1.3 深度学习训练系统内存行为

为了减少 DNN 训练中 GPU 内存的占用量,我们必须了解深度学习训练系统的内存行为。本小节通过对典型深度学习网络训练过程中的内存行为进行记录和分析,对深度学习训练系统的内存消耗类型,迭代特征和内存碎片情况进行介绍。整体上,我们可以看到,模型特征图为 DNN 训练过程中的主导内存消耗类型,而训练的不同迭代之间在访存行为上保持高度一致,虽然目前的深度学习编程框架使用了内存池技术对池内数据做了内存复用的设计,但是在实际训练过程中还

是存在数量可观的内存碎片。

关于内存消耗类型。我们对具有六个代表性 CNN 的 GPU 内存占用情况进行了研究。图 1.5 中显示了 GPU 内存占用量的细分。可以了解到，GPU 内存往往被特征图占据。例如，Inception V4 需要 90% 以上的 GPU DRAM。此外，将这些特征图保留在 GPU 内存中，即使它们具有未来但又很遥远的依赖关系，这也是一种浪费，特别是对于非常深的神经网络而言。因此，中间特征图是在 DNN 训练中优化内存使用的关键因素。下面针对这几类内存类型进行详细分析。

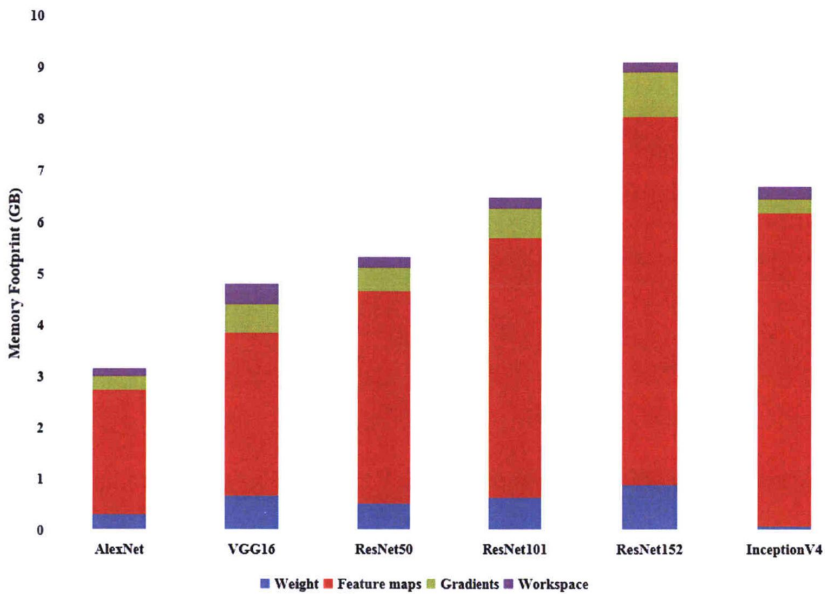


图 1.5 不同 DNN 训练内存占用分解

Figure 1.5 The breakdown of memory footprint in DNN training for different networks

特征图。特征图是深度学习计算所必需的，包括了前向计算中的特征图和反向传播中的梯度图。在前向传递中，输入特征图通过 DNN 向前馈入当前层。每个隐藏层都接受输入数据，按照激活函数对其进行处理，然后将其输出传递给后续层。在前向传递的末尾，网络获得损失函数值，然后计算梯度以在后向传播期间更新权重。在前向传播中生成的特征图通常驻留在设备内存中，直到用于计算梯度为止。在向前传播期间，整体内存使用量将增加。梯度图是在后向遍历中生成并由相关层消耗的中间结果。图 1.6 中显示了 ImageNet 数据集上 10 次小批量训练迭代的 ResNet18 内存负载的曲线。随着特征图的积累，曲线的最大值逐渐出现。特征图的大小通常由批处理大小和模型体系结构确定（对于 CNN，步幅大小

和输出通道数，对于 RNN，则是门数，时间步长和隐藏大小）。不再需要的特征图将被释放，从而导致曲线下降。对于复杂的模型训练，用户必须调整批处理大小，甚至重新设计模型架构，以解决“内存不足”问题。尽管使用模型并行性，可以将一项训练任务拆分到多个设备上以缓解此问题，但这带来了额外的通信开销。而且设备之间的带宽限制可能会大大减慢训练过程。

模型参数。 在前馈推理阶段，参数（也称为权重）通常是只读的，其值决定了验证样本对输出的影响。训练阶段包括两个阶段，即前进和后退。具体来说，在后向阶段，后向传播算法将传播误差并计算可用于调整参数以提高精度的损失函数的梯度。在训练阶段，参数在每次迭代结束时更新，并在下一次迭代中使用。对于给定的 DNN，参数的内存消耗通常与层数（深度）和层大小（宽度）成比例。与功能图相比，模型参数在内存使用中所占的比例相对较小。通常在一次训练迭代中驻留在 GPU 内存中。

临时存储。 对于某些算法，例如基于快速傅立叶变换（FFT）的卷积^[12]，许多操作都需要额外的内存。它是临时的，将在操作过程中释放。可以通过枚举诸如 cuDNN^[13]之类的 GPU 软件库中的每种算法来自动调整临时存储器的大小，因此可以忽略它。

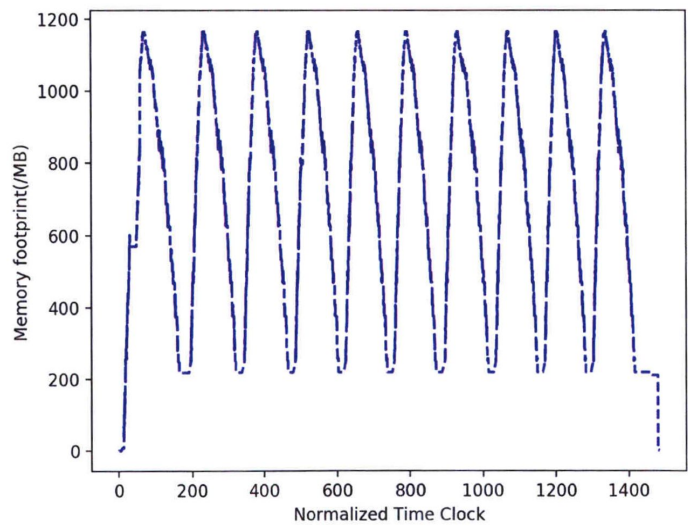


图 1.6 ResNet18 10 次迭代内存负载曲线

Figure1.6 Memory load curve for 10 iterations of ResNet18 training

关于训练的迭代特性，许多机器学习算法（例如 DNN）都是通过迭代的方式

优化模型。用于训练 DNN 的随机梯度下降 (SGD) 算法通常需要数千次迭代 (甚至更多次) 才能收敛。在每次迭代中, 网络都会进行正向传播和反向传播, 如图 1.5 所示, 在此期间, 将创建, 读取, 更新和删除多达数万个大小不同的变量。网络在正向传播期间计算特征图, 即 $h_1, h_2, \dots, h_{n-1}$, 在正向传播结束时获得损耗, 然后计算梯度以在向后传播期间更新权重。在前向传播中计算出的特征图通常会保留在 GPU 内存中, 直到用于计算权重梯度为止。因此, 整体内存使用率在正向传播期间增加, 在正向传播结束时达到峰值, 而在向后传播期间减少。ResNet18 网络的 10 次迭代训练过程中的内存负载变化曲线如图 1.6 所示, 每个操作索引都引用一个 malloc / free / read / write 操作。可以看到每一次迭代的内存负载都是稳定的, 且在不同迭代之间保持不变。这种迭代的性质使我们能够在开始的迭代中收集变量的生存期和读/写语义, 并为后续的迭代优化内存管理。

为了精确指出 GPU 每个内存块的行为, 我们通过检测运行时系统的所有定位器, 从而仔细表征 DNN 工作负载的内存行为。假设我们要在 Nvidia Titan RTX GPU 上训练 Resnet18。我们已经在 ResNet 工作负载训练中绘制了前十轮迭代的甘特图, 如图 1.7 所示。图的 x 轴是归一化的时间戳, y 轴是 GPU 的归一化存储器地址。图中的每个矩形代表对 GPU 的设备内存中某个内存块的内存访问行为。矩形的宽度是从分配时间到当前存储块的空闲时间所经过的时间, 可以视为当前存储块的寿命。矩形的高度表示当前存储块的大小。显然, 我们可以从图中看到, 在 ResNet 工作负载的训练过程中, 不同迭代之间, 同一内存块的分配位置是一致的, 也即不同迭代中, 同一内存块的摆放位置在训练过程中保持不变。因此可以知道, 前十轮迭代中存在稳定的内存访问模式。除此之外, 蓝色的矩形中间会出现白色的空当, 这些都是 GPU 内存中存在的碎片, 碎片的数量可观, 这些碎片降低了 GPU 的内存利用率。

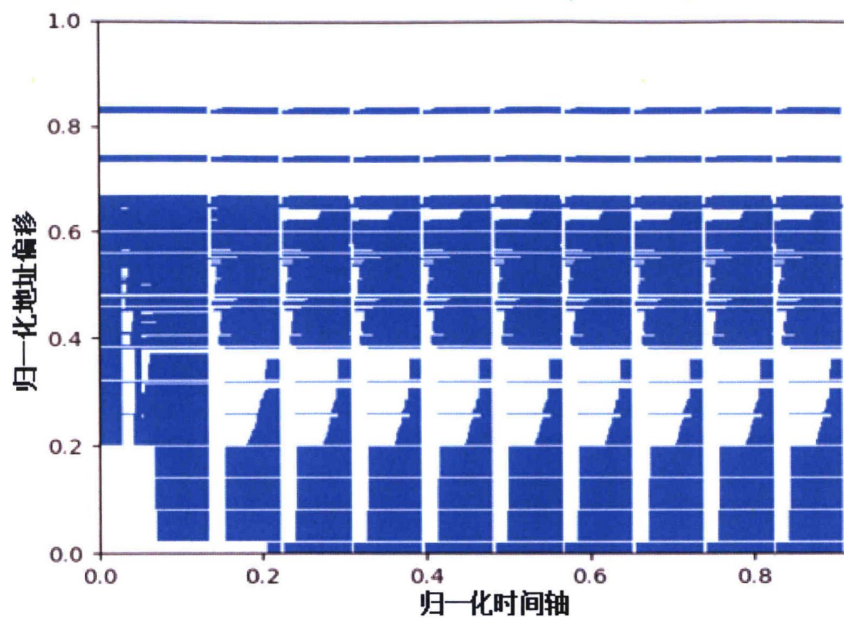


图 1.7 ResNet 前 10 次迭代甘特图

Figure1.7 Gantt chart of the first ten iterations in ResNet training

1.1.4 内存使用低效问题定义

针对深度学习训练系统中内存行为的特性，可以发现目前 GPU 内存不足的原因有两个：

首先是 DNN 训练中数据的复用特征和其在网络中的位置密切相关，前几层数据的复用将跨越一次迭代的整个过程，现有的深度学习计算系统难以充分地利用这些特征，从而使得数据长期驻留，造成了内存使用的低效。

其次，现有的深度学习训练系统通常提供了通用的运行时内存分配器，这些内存分配器都是基于动态内存池实现的，不能充分地利用深度神经网络架构相对确定的全局信息，容易出现内存碎片。

为了解决部分数据长期驻留带来的低效率问题，可以将访问间隔大的内存块在空闲的区间内拷贝到 CPU 主存，从而降低内存负载的压力，并减少内存占用，这样的过程称为 Memory Swapping 内存交换。内存交换策略的优劣对于运行时开销的影响很大，为了在降低内存负载的同时尽可能地减少运行时开销。就必须仔细考虑交换内存块的选择以及内存交换事件的调度。因此，如何高效利用 GPU

内存，设计合理的交换内存块（swapblock）选择机制和内存交换事件调度策略，来有效地降低内存负载并且保持尽可能低的运行时开销，是深度学习训练系统内存优化中一个亟待解决的问题。

为了解决内存碎片带来的低利用率问题，可以借助 DNN 训练中稳定的迭代特征。与动态内存分配问题不同，在 DNN 训练场景中，每一次迭代的访存行为和内存占用情况都是一致的，因此在一次迭代中的全部内存分配和释放的请求都是确定的，这就使得从全局角度优化内存分配成为可能。在已知一次训练迭代中所有内存块的生命周期和大小的前提下，如何编排优化这些内存块的分配位置，使得整体内存碎片大幅度降低是解决内存碎片问题的关键。此外，为了保证 DNN 训练的运行时效率，优化内存分配问题的算法不能有过高的复杂度，研究一个启发式算法得到合理的近似解也是亟待解决的问题。

1.2 本文的主要贡献

针对上述的两个问题，本文研究的主要内容专注于 GPU 内存复用和 GPU 内存交换两个问题，这两个问题彼此是相互正交的，并且对应于解决深度学习训练过程中产生的内存利用率低和内存使用低效问题。本文的主要贡献如下：

首先，本文提出了内存高效的深度学习训练系统 METS（Memory Efficient Training System），利用深度学习训练的迭代性，自动提取前几轮迭代的 profile 信息，并基于 profile 信息制定自动内存交换策略和全局内存分配优化策略。通过自动内存交换提高内存使用效率，并且使用全局内存分配优化模块提高内存利用率。

其次，本文提出代价模型的自动内存交换。内存交换是为了让在训练过程中缺失时间长且较大的内存块在不被使用的时候被交换到 CPU 主存，并在下一次访问之前再交换回 GPU。针对深度学习训练过程中的迭代特性，我们可以通过对前几次迭代的特性进行分析，总结和归纳出一个代价模型，进而帮助内存交换模块选择在何时将哪一个内存块交换出去，同时最大限度降低运行时的开销。

最后，本文提出了基于全局访存行为的内存分配优化策略。由于动态内存分配器在处理内存分配释放请求的过程中，不能掌握全局的信息，所以容易导致内

存碎片的产生。因此我们通过统计前几轮迭代过程中各个内存块的大小和生命周期，得到各个内存块的相互冲突关系，从而在一次迭代的全局信息基础之上优化每一个内存块在内存池内的摆放。从而使得 Tensor 在内存池内的摆放更加紧凑，进一步的压缩内存碎片的数量。此外，卷积算子的 Workspace 在 GPU 中短期驻留，其在执行算子之前被分配，在算子执行完成之后被释放。这样的访问模式适合被动态分配和释放，因此这里设计了双端分配策略，将卷积算子的工作空间由高地址向低地址分配，其余内存块通过全局分配优化策略选择摆放地址。

1.3 论文的组织结构

本文由 6 个章节组成，各个章节的内容结构如下：

第一章简要描述了深度学习的训练场景，包括了深度学习训练的过程介绍，深度学习训练系统的内存行为特征，并提出了深度学习训练中存在的内存使用低效率和低使用率的问题，进而引出本文的主要研究内容和最终设计的内存高效训练系统 METS (Memory efficient training system)。

第二章介绍深度学习训练中国内外的研究现状，分四个方面对现有的内存优化技术和解决方案进行介绍。在现有研究现状的基础上给出本文技术方案和不同之处和创新点。

第三章介绍 METS 的整体结构和运行流程。主要包括了两个核心优化技术，基于代价模型的自动内存调度技术以及基于全局访存行为的全局内存分配优化技术。

第四章介绍基于代价模型的自动内存调度技术。包括了内存自动拷贝的开销建模，代价模型构建，内存调度策略的设计方法和实现原理。

第五章介绍基于 Profile 的全局内存分配优化技术。包括二维条带打包问题的介绍，小规模问题的整数线性规划解法，大规模模型训练问题的启发式解决方法。对于短生命周期的内存进行动态分配和释放的优化策略。

第六章介绍本文的实验测评与结果分析。首先对实验的配置，实验所涉及的硬件环境和软件环境进行介绍，其次介绍了实验的测试集和实验时的相关参数配置。最后是对各个实验结果的多角度分析。

第七章是全文的总结和展望，对本文的主要内容进行总结并提出下一步的工作方向。

最后是参考文献和致谢的部分。

第 2 章 相关研究

目前, 工业界和学术界已经提出了许多减少深度学习训练系统内存占用的方法, 本章将工业界深度学习框架的内存解决方案和学术界的最新进展分开分析。从应用方法的角度上看, 内存优化的方法可以分为四个方面, 包括模型压缩, 内存共享, 内存交换和输出重计算。本章将对以上的解决方案进行介绍, 并阐述本文方法与上述方法的不同之处。

2.1 深度学习框架内存解决方案

2.1.1 异构系统简介

深度学习训练系统通常是一个异构系统, 由主机 CPU 和 GPU 组成。GPU 通常通过高性能总线 (例如 PCIe 和 NVLink) 连接到主机 CPU。主机 CPU 通常扮演控制角色, 例如启动内核和线程调度。GPU 负责与 DNN 相关的任务的计算, 例如矩阵乘法, 卷积, 池化和激活操作。在执行 GPU Kernel 之前, 需要将输入数据驻留在 GPU 内存中。GPU 的供应商提供了高级编程 API, 以管理设备内存和执行命令, 并在主机 CPU 和设备加速器之间来回交换数据。CUDA 程序员可以使用 `cudaMalloc/cudaFree` 在 GPU 上 `malloc` / 释放设备内存块, 并使用 `cudaMemcpy` 在 CPU 和 GPU 之间交换数据。显卡供应商还提供异步 API 以重叠 I/O 和执行, 例如 `cudaStream_t`。

2.1.2 动态内存池简介

内存管理是计算机系统 (包括数据库系统) 的重要组成部分。已经提出了各种技术, 例如分页和缓存优化。**动态内存池技术**是有效处理 DNN 训练过程中内存复用的技术手段。它预分配了连续的内存块, 并从操作系统接管了内存管理。为了获得较小的时间复杂度, 内存池实现有多种变体。CnMem^[14]是 Nvidia 开发的 GPU 内存池。但是, 它没有针对深度学习训练的优化。在本文中, 我们建议通过利用变量的生存期和大小信息来优化 GPU 内存池中的内存分配, 从而以较低的时间复杂度实现更低的内存碎片。

许多深度学习框架（例如 Theano^[15]，TensorFlow^[16]和 Chainer^[17]）基于内存池动态分配 GPU 内存，内存池维护两个链表和垃圾回收机制，一个链表维护已使用的内存块，另一个链表维护未使用的内存块。在给定 GPU 内存请求的情况下，动态内存池会从未使用链表中找到足够大的最小内存块，如果没有找到这样的块，则从向物理内存申请分配指定大小的内存来满足这一请求，并用引用计数包装该块。当内存块的引用计数为 0 时，它将被回收，并返回到内存池中未使用的链表内。

2.1.3 深度学习框架内存减少技术

在当今的深度学习框架中，已采用了几种内存减少技术，以解决 GPU 内存短缺问题。Caffe^[18]和 Torch^[19]直接将前向数据张量重用于后向数据传播，这可在线性网络上节省多达 50% 的内存。尽管此技术在线性网络上效果很好，但它需要额外的张量来保存将来对非线性网络训练的依赖性，从而限制了有效性和效率。而且，这种内存重用的程度对于现代深层非线性神经网络来说是不足够的。MXNet^[20]和 TensorFlow 使用有向非循环图（DAG）执行引擎构建。用户明确定义了计算流程和张量依赖关系，这为 DAG 引擎提供了分析张量寿命的必要信息。然后，两个系统都释放了不再需要的张量以节省内存。MXNet 还实现了基于每层的重新计算策略，该策略类似于 Spark^[21]中的弹性分布式数据集（RDD）。具体来说，它释放前向遍历中由计算廉价层产生的张量，并通过执行另一个前向来重新计算释放的依赖关系以用于后向遍历。换句话说，这个方法以牺牲速度为代价，积极地减少了 GPU 内存的使用；但是其忽略了网络层的不均匀内存分配，从而顺序地产生大量不必要的内存使用。

现有的深度学习框架没有高效率地利用 GPU 内存，需要用户减少训练的 BatchSize 来适应 GPU 物理内存的限制，当新的内存请求不能被满足时，就会出现 OOM(out of memory)。上述深度学习框架均未使用内存交换策略，该策略可以为深度学习网络训练提供更多的内存空间，提高内存的使用效率，并且可以在给定现有 GPU 内存资源的情况下优化训练速度。此外，动态内存池技术由于为了适应各种不同上层应用的计算要求，**不能预知全局的内存请求和释放情况**，未使用的内存块只有在相邻的情况下才能被融合为更大的内存块。因此当新的内存请

求来临时,虽然有足够大小的未使用内存块,却无法满足不同内存请求的需要。总之,这些挑战促使我们设计内存交换策略和使用全局内存行为优化内存分配,以提供训练所需的内存,同时合理分配卷积工作区的内存以优化训练速度。

2.2 深度学习训练内存优化的研究现状

目前已经有很多工作致力于降低单个设备训练神经网络时的内存消耗。这里从深度学习训练中内存优化的技术的角度对当前一些代表性工作进行分类阐述,并对其优缺点进行分析。

2.2.1 模型压缩

模型压缩通过降低参数的精度或网络结构的复杂性来降低深度学习中的内存负载。半精度(16FP),单比特和混合精度数字已在DNN中应用。通常,使用低精度数字会引入量化误差。删除DNN中的层或连接的模型修剪,降低了复杂性和内存负载;但是,这也会降低训练精度。

典型的模型压缩技术包括修剪和量化。

模型修剪基于以下事实:参数中存在大量冗余。修剪技术尝试在不损失准确性的情况下修剪权重。结果,性能和内存使用率均得到改善。例如 Han 等^[25]人修剪了网络模型的连接。

参数量化,即通过减少用于表示每个参数的位数来压缩DNN。Gong 等^[26]人,2014年以保持准确性的方式量化可学习的参数,一些研究甚至鼓励使用二进制数来提高DNN工作负载的性能,这也带来了内存使用率的节省。

我们在图1.5中的统计数据表明,对于典型的DNN工作负载,参数仅占用很少的内存消耗,这表明权重修剪或量化技术在减少训练中的内存占用方面并不有效。此外,这些模型压缩技术通常会导致精度损失,这在DNN域中是不可忽略的。压缩的另一个问题是需要耗时的重新训练。

模型压缩方法是对我们方法的补充,但主要区别在于压缩会更改模型的行为,因此压缩模型可能无法按网络设计人员的预期工作。另一方面,我们的方法不会更改模型所涉及的计算。

2.2.2 内存共享

对于深度学习框架，已经提出了两种类型的内存共享，它们是 **In-place** 操作和跨层内存重用。

In-place 操作是将输出存储在输入的物理地址，将相同的内存区域重用于输入和输出。例如，在计算 $y = \text{sigmoid}(a)$ 时，可以将 y 直接存储在 a 的存储块中，实现 y 和 x 共享存储空间。DNN 中只有少数几个 **In-place** 操作的机会。如 MXNet-memonger 中所实现的，内存重用是在寿命不重叠的变量之间共享内存。它可以方便地在声明式范式的深度学习框架中实现（例如 TensorFlow, MXNet），在该框架中，整个计算图是在计算之前构建的，从而提供了拓扑和数据相关性，以实现更智能的内存分配。

跨层内存重用中最具代表性的方案是基于层融合的解决方案，例如 TASO^[27]。这些技术试图通过图替换用某些融合块替换原始计算图中的结构化块。对于其寿命不重叠的融合块中的那些变量，还有一些重用机会。层融合技术趋向于去除跨层的临时变量，从而减少内存占用。

应该注意的是，本文提出的基于全局访存行为的内存分配优化算法提供了更好的（**更细粒度的**）**内存共享**，使得内存块的分配更加紧凑，内存碎片更少。这将在第五章中详细介绍。

2.2.3 内存交换

内存交换方法本质上是利用 CPU 主存来扩展 GPU 设备内存。基于交换的方案并非总是将数据保留在设备内存中，而是采用了数据卸载和预取的思想。不用的数据被交换到主机存储器中，并且在需要它们之前被交换回 GPU 中。为了减少内存交换的拷贝开销，应该通过单独的异步流将双向隐藏在计算中。理想情况下，双向通信应隐藏在计算中（通过单独的流）以最小化通信开销^[27]。

类似的虚拟内存技术在 CUDA 8.0^[28]中已经有实现，具体来说是通过页面迁移引擎启用了统一内存，因此统一内存不受 GPU 内存大小的限制。但是，我们的测试表明，它会带来严重的性能损失（最大 10 倍的降级）。另一种方法是使用运行时内存管理器通过卸载每一层地输出并在必要时预取它来虚拟化内存使用情况，该方法只能应用于分层 CNN 模型，不能应用于序列模型。

在 GeePS^[29]中, 待交换的层或张量由用户决定。它要求最终用户对模型有很好的了解, 包括每层的内存和时间消耗。vDNN^[30]采取了蛮力策略, 并且仅卸载了所有层或所有卷积层的输出。vDNN ++^[31]提出了一种近似的内存池, 以减少 GPU 内存碎片。SuperNeurons^[32]限制仅交换卷积层。其他层的大张量不考虑交换。它还需要 DNN 的计算图或最终用户的干预。TF-LMS^[33]提出了一些形式化的图形重写规则, 以便在计算图上插入换入或换出节点。SwapAdvisor^[34]基于 DNN 工作负载的计算图, 沿着三个维度执行联合优化, 包括 Op 调度, 内存分配和交换张量决策。它还提出了一种基于启发式的定制设计遗传算法, 以探索广阔的搜索空间。

本文期望提供一种**对最终用户透明并且能够以小的通信开销交换内存**的全自动调度方法。

2.2.4 输出重计算

输出重计算方法的思想是使用计算来换取内存。这些方法会在 GPU 内存不足时丢弃一些计算结果, 并在需要时重新计算它们。一些中间变量(例如特征图)在正向传播期间释放, 直到在向后传播阶段中需要它们时, 重新计算这些中间结果, 用于计算梯度。

BPTT^[35], SuperNeurons, Capuchin^[36]和 KARMA^[37]也采用了类似的想法。它们都使用经验成本模型来选择性地删除重新计算便宜的特征图。根据 DNN 工作负载的计算图, 经验成本模型与层结构或计算图语义紧密耦合。由于某些层的重新计算时间较长, 这些方法可能会导致性能明显下降。这种方法可能适用于那些线性网络(在 DNN 的层结构中没有分叉和连接, 例如 AlexNet 和 VGG), 因为在线性神经网络层之间没有复杂的重新计算依赖性。但是对于那些在计算图结构中具有多个 Fork 和 Concat 的非线性 DNN, 各层之间复杂的执行依赖关系可能会引入复杂的重新计算依赖关系。对于那些新出现的具有复杂层结构或复杂计算图的 DNN, 情况甚至可能进一步恶化。

还有其他一些优化方法, 例如内存有效的 DenseNets^[38]和内存有效的卷积, 可显著减少内存消耗。但是, 高效内存的 DenseNets 适用于特殊的模型, 高效内存卷积尽管可以减少卷积计算中的临时内存, 但是这部分内存与特征图相比, 可以

忽略不计。

2.3 本章小结

通过对工业界的编程框架和学术界的解决方案进行归类分析。我们强调了现有方案在减少深度学习训练的内存压力方面的局限性。模型压缩技术可能会导致准确性下降，这在深度学习领域中不可忽略。需要大量的微调工作来弥补精度损失。输出重计算技术不可避免会带来性能的开销，这些开销不是一次性的，而是均匀地分布在训练迭代过程的始终，对训练场景来说影响巨大。本文的方法不涉及以上两种技术方向，且与以上两种技术方向正交。

现有的内存交换技术一方面与上层算法的计算图结构紧密耦合，另一方面主要借助用户自定义的规则进行内存的交换。我们期望提出一种对算法部署和研究用户完全透明的，全自动的解决方案，该方案能够摆脱用户对内存交换策略的干预，并且以较小的运行时间开销解决 DNN 训练的负载压力问题。

现有的内存共享，主要借助层融合和跨层内存重用来提高内存的利用率，一对一的内存复用存在局限性。首先是层融合依赖于计算语义的分析，对于全新的模型不友好，其次是动态内存池无法预知全局内存分配和释放情况，不可避免地带来内存碎片。因此我们期望给出一种与上层计算语义无关的全局内存分配优化算法，利用深度学习训练的迭代性，从 Profile 中得到全局的分配和释放信息，利用全局信息优化内存块的摆放，进一步的压缩碎片，从而降低训练中的内存占用。

第3章 内存高效训练系统总览

为了解决深度学习训练系统中内存利用率低和内存使用效率低的问题。本章将介绍内存高效训练系统 METS (Memory efficient training system)，一个用于训练深度神经网络的动态 GPU 内存管理运行时系统。允许算法研究者探索超越 GPU 内存物理限制的更深，更广泛的模型，或者是以更大的 batchsize 训练模型。它使用张量（内存池中内存块）作为基本的调度和分配单元。运行时无缝地协调张量的放置，交换，分配和释放，最终底层的内存操作对用户完全透明。METS 的系统结构如图 3.1 所示。

3.1 METS 系统结构

METS 是一个内存高效的全自动深度学习训练系统。其主要目的是提高深度学习训练系统的内存利用率和使用效率，并有效地减少 GPU 内存消耗。为了实现此目的，METS 利用训练 DNN 的迭代性质，开发了内存行为分析模块用以从前期迭代中获取训练迭代的全局信息，并根据 DNN 负载的访存模式设计和实现了两个正交模块分别解决内存利用率和使用效率的问题。这两个模块分别是基于代价模型的自动内存交换模块和基于全局访存行为的内存分配优化模块。自动内存交换可以在 CPU 和 GPU 之间自动交换内存，减轻 GPU 的内存负载压力以提升内存使用效率，而全局内存分配优化可以对张量的分配进行管理，合理安排内存块的摆放位置，压缩内存碎片以提升内存利用率。这两种技术相互正交，并且不需要人工干预。

内存行为分析模块主要在 DNN 负载训练的前期数次迭代中自动检测内存行为并获取稳定的内存访问模式，提供给后续模块使用。后续模块的优化策略生成之后被应用在剩下的所有训练迭代之中。

自动内存交换模块用于提高深度学习训练系统的内存使用效率。通过在 CPU 和 GPU 的物理内存空间之间交换张量，有效地降低深度学习训练中的内存负载峰值，将重用距离长的内存块交换到 CPU 主存，从而降低了 GPU 内存负载的压

力。现代 GPU 配备了独立的直接内存访问（DMA）引擎，从而暴露了在计算中隐藏通信的机会。通过分析数据拷贝的开销，在给定的内存负载峰值约束下，METS 精心地将通信与计算重叠。在宽松的内存负载限制下，通信和同步可以完全隐藏在计算之下。但是，在严格的内存负载限制条件下，由于无法快速换出变量，因此内存负载将超过限制。如果它在正向传播期间发生，则下一个 Malloc 将被延迟，直到交换出足够的张量并释放内存为止；如果在向后传播期间，只能在释放一些张量后才能开始交换。这是通信和同步开销发生的地方，并且无法隐藏。

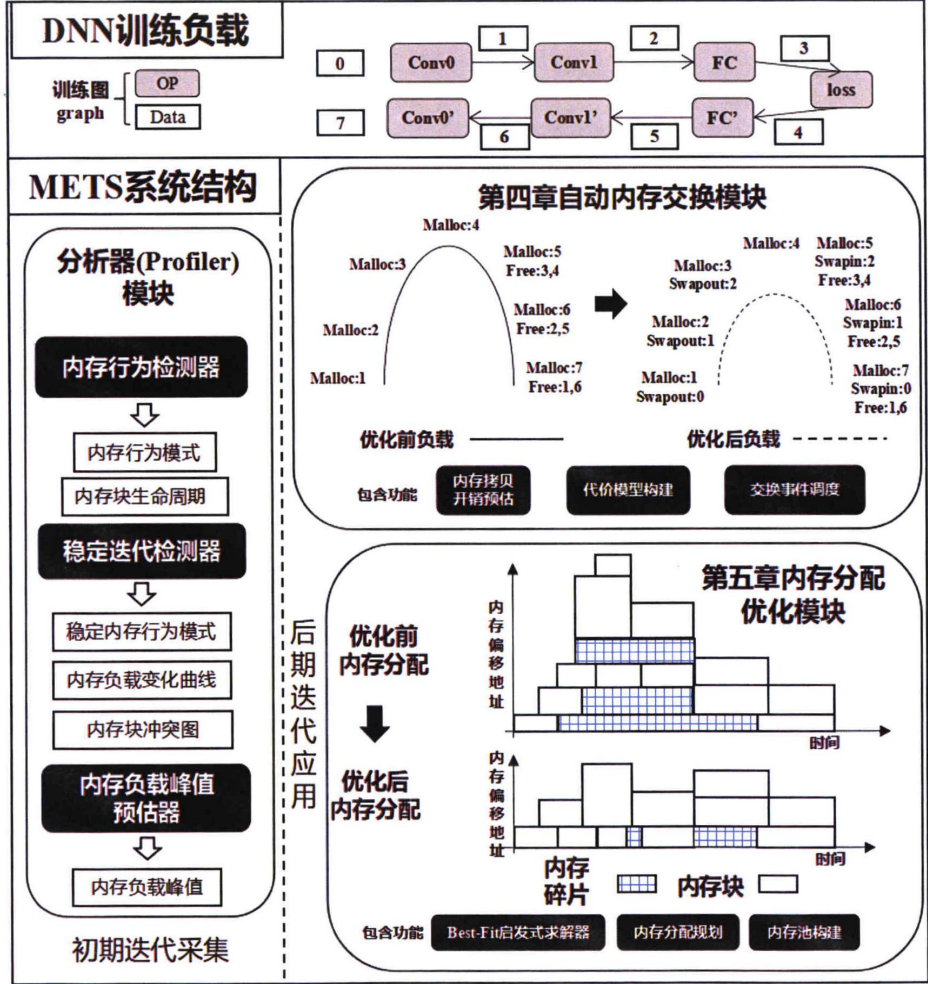


图 3.1 METS 系统结构示意图

Figure3.1 The architecture of memory efficient training system

内存分配优化模块用于提高内存的利用率。通过合理地安排内存块的摆放位

置,最大限度使张量内存块相互复用来减少内存碎片的产生。GPU 需要分配内存来创建张量,并需要释放内存以释放张量。由于动态分配内存池无法预知全局内存分配和释放信息,在内存请求复杂多变的情况下,往往会出现内存碎片。我们提出了一种基于全局内存块冲突图的分配优化算法,给一次迭代的每一个内存块分配一个合理的全局绝对地址,并且应用在后续每一次迭代中。

3.2 METS 系统工作流程

METS 包括了两种内存优化技术，内存交换模块可以有效降低内存负载峰值，减轻内存的使用压力，内存分配优化模块可以降低内存碎片，提高内存利用率。最终都起到了降低训练过程中总体内存占用的效果。随着内存占用量的降低，用户可以在 BatchSize 和模型复杂度之间权衡。也即，相同模型用户可以使用比优化前更大的 BatchSize，或是在相同 BatchSize 的前提下，使用更深更宽的模型进行训练。图 3.2 展示了 METS 的工作流程。

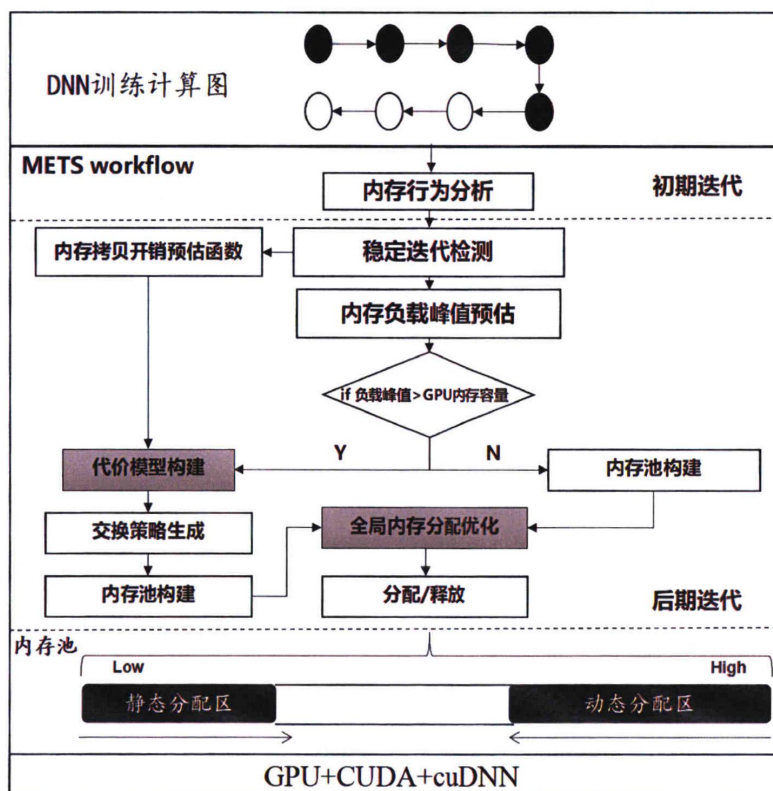


图 3.2 METS 工作流程

Figure 3.2 The workflow of METS

METS 首先会打开 CUDA unified memory, 确保模型可以获得比物理内存更大的空间, 在开始的迭代中对内存行为进行检测, 并提取出稳定的内存行为序列。由一次稳定迭代的内存行为序列中, 可以预估得到训练中所需内存负载的峰值, 如果这一峰值高于 GPU 内存容量, 则必须进行内存交换。内存交换模块为了在尽可能小的运行时开销下降低内存负载峰值, 必须对交换内存块做精心的挑选, 并设计好内存交换事件的执行顺序。这些决策都需要代价模型的帮助, 代价模型为了计算内存块的缺失时长收益, 需要获知内存块从 CPU 拷贝到 GPU 和从 GPU 拷贝到 CPU 的时间开销。因此系统必须针对当前模型给出准确的内存拷贝开销函数。在内存交换策略生成之后, 系统需要更新 Profile 的行为日志, 并构建内存池。由于 Profile 中可以轻松得到内存块的生命周期及其重叠关系。据此系统可以给出全局内存分配优化方案, 确定各个内存块的分配地址和复用策略。由于 DNN 训练的迭代性, 上述的复用策略和内存交换策略可以被应用部署在后续的迭代中, 而不会改变训练的准确性。

3.3 METS 系统模块介绍

METS 可以被灵活的嵌入在开源深度学习框架中, 输入为深度学习模型的训练源代码, 深度学习框架完成计算图的构建以及算子的调用过程, 在内存管理的部分由 METS 负责完成。

METS 由三大模块构成, 分别是内存行为分析模块, 自动内存交换模块以及全局内存分配优化模块。图 3.1 展示了 METS 的系统结构。内存行为分析模块负责从前期的训练迭代中提取内存的访存模式和动态负载情况。自动内存交换模块完成训练过程中的自动内存交换, 包括交换内存块的自动选择, 内存交换事件的调度。全局内存分配优化模块负责为一次迭代中的所有内存块找到合适的摆放位置, 使得内存块的摆放更加紧凑。随着内存占用量的降低, 用户可以使用比优化前更大的批处理大小, 或是在相同批处理大小前提下, 训练更深更宽的模型。

3.3.1 内存行为分析模块

内存行为分析(Profiler)模块是后续优化技术的实现基础, 这里称为 Profiler 模块, 主要负责对训练前期迭代中的内存行为进行分析, 得到内存访问模式和动态

负载信息，以便自动内存交换模块和内存分配优化模块利用。内存行为分析模块是很有必要的；一方面因为自动内存交换模块需要做待交换的选择以及交换事件的调度，就必须了解运行过程中内存块的访问间隔，内存大小，动态内存负载情况以及各个内存块被访问的时间顺序。另一方面，全局内存分配优化模块必须明确各个内存块生命周期的重叠关系才能构建内存块的冲突图，并进而对内存块的复用关系进行优化，以使有限的存储空间被尽可能地复用。Profiler 模块包括三个功能：内存行为检测器、稳定迭代检测器和内存负载峰值预测器。分别完成初期迭代的内存行为信息采集，稳定行为序列检测和内存负载峰值的计算。

1) 内存行为检测器

内存行为检测器负责检测和记录训练过程中的每一次访存行为。在框架的运行系统中插桩，并自动检测和跟踪 DNN 训练过程中每一个内存块的行为日志，如图 3.3 所示，包括申请(MALLOC)、访问(ACCESS 包括读取、写入)和释放(FREE)的操作内存块地址，内存块大小，以及对应的时间戳。其中，申请和释放行为可以从 TensorFlow 的内存分配器中插桩得到，而内存访问记录可以从 TensorFlow 核心源码的 Tensor 类的 data 函数中插桩得到。

```
...
MALLOC: 0x7f75bd800000, 37632, 1613919615127465296
ACCESS: 0x7f75bd800000, 37632, 1613919615127666623
MALLOC: 0x7f75bd809400, 256, 1613919615129325904
ACCESS: 0x7f75bd809400, 256, 1613919615129410286
ACCESS: 0x7f75bd809400, 256, 1613919615129420285
ACCESS: 0x7f75bd800000, 37632, 1613919615129430285
FREE: 0x7f75bd800000, 1613919615129430305
FREE: 0x7f75bd809400, 1613919615129430400
...
```

图 3.3 Profile 数据格式

Figure 3.3 Profile data format

内存行为检测器在模型训练的初期迭代检测和记录访存行为。为了尽可能地降低内存占用，找到模型训练的最低内存需求量，系统会在前行计算的每一层后插入 Swapout 操作，并在反向传播的每一层前插入 Swapin 操作，从而保证使用最少的内存。如果 GPU 内存容量不能满足这一执行模型下的运行，则模型不可训练。关于 OP 的执行顺序，不在本文的讨论范围内。

2) 稳定迭代检测器

由于在 DNN 训练启动的初期，系统需要在 GPU 上完成一系列资源初始化操作，训练前几次迭代的内存行为并不稳定，导致最初迭代的内存负载行为与后续迭代并不完全相符，而 profile 需要被应用到后续迭代之中，因此 profile 中的访存行为日志必须与后续迭代完全一致。为达到此目的，结合目前常见应用的经验分析，我们设计稳定迭代检测器从 5 次迭代中检测到稳定的一次迭代行为序列，作为后续优化的 profile 信息。

3) 内存负载峰值预测器

内存负载峰值预测器预测出模型训练所需的最小内存空间，如果内存容量低于这一阈值，则无法在当前配置下正常训练模型。由于 Profiler 的加入，自动内存交换模块可以获得计算内存交换的收益和代价的必要信息，内存分配优化模块能够提取预知全局的访存信息，这些重要的数据在后续迭代中不会发生改变。因此系统的优化不需要用户的手工干预，而是从 profile 信息中全自动地生成内存优化方案。如图 3.1 所示，METS 对上层应用有良好的适应度，用户不需要为了内存降低做手工的优化和配置，后续的内存优化技术也不会受到具体模型的影响。

给定模型训练第一次迭代中的访存行为序列为 A，

$$A = \{a_0, a_1, a_2, \dots, a_{n-1}\} \quad (3.1)$$

A 中每一个元素都代表一个访存行为，可能是分配、释放或访问。

$$a_i \in \{\text{alloc}, \text{free}, \text{access}\} \quad (3.2)$$

系统会根据式 3.3 计算每一个访存行为过后内存负载的动态值，其中 s_i 为当前访存行为访问内存块的大小。 α_i 的值由访存行为的类型来决定，当访存行为是分配，其值为 1，当访存行为是释放，其值为 -1，若访存行为是读写，其值为 0。

$$\text{MLoad}_k = \sum_{i=0}^k \alpha_i * s_i \quad (3.3)$$

$$\alpha_i = 1 \text{ iff } a_i = \text{alloc} \quad \alpha_i = -1 \text{ iff } a_i = \text{free} \quad \alpha_i = 0 \text{ iff } a_i = \text{access} \quad (3.4)$$

$$M_{\text{peak}} = \arg \max_k \text{MLoad}_k \quad (3.5)$$

每一个访存行为后内存负载动态值获取之后，内存负载峰值即可以通过求解

最大的动态负载值来得到。这一内存负载峰值将决定当前 DNN 训练负载是否可训练，若 GPU 内存容量低于这一峰值，则模型不可训练。

3.3.2 自动内存交换模块

自动内存交换模块自动将未使用的内存块从 GPU 交换到 CPU 内存，并在下次访问之前将它们交换回去。通过第二章可以了解到，用于训练 DNN 的反向传播算法具有一种特殊的模式，即仅在每次迭代的开始和结束时访问底层的变量。因此，交换方法具有显著节省 GPU 内存的潜力。从功能上看包括内存拷贝开销预估、交换内存块代价模型以及交换事件调度三个函数。

内存拷贝开销预估函数用于计算给定大小内存块的拷贝延迟。由于内存交换会带来运行时的开销，为了自动确定需要被交换的内存块，系统必须知道每一个内存块交换的准确拷贝传输开销，以便于估计内存块交换的代价和收益。

交换内存块代价模型主要用于交换内存块的自动挑选。这里通过代价模型来辅助内存块的自动挑选，可以直接计算内存块的细粒度收益，并通过对内存块的收益进行排序，确定内存块交换的次序。

交换事件调度合理地安排内存交换事件的执行顺序，从而保证自动内存交换带来的运行时开销尽可能小。

更具体的介绍在本文第四章。

3.3.3 全局内存分配优化模块

全局内存分配优化模块实现了一个高效 GPU 内存池，该内存池基于所有变量的生存期来优化内存分配。例如，寿命没有重叠的变量可以被分配到相同的空间，即内存共享。但是，找到最佳分配方案是一个 NP 难问题。在本模块，提出了一种启发式算法来解决它。

根据 profile 中得到的内存行为日志，系统可以检测出各个内存块的分配时间戳和释放时间戳，进而可以掌握一次训练迭代中每一个内存块的生命期及其大小。内存分配优化模块通过生命周期的冲突关系得到全局内存块的冲突图，在这一冲突图基础上求解全局内存分配优化问题，从而实现全局内存占用最小化。这一问题可以通过 MILP 来求解得到最优解，也可以使用 Best-Fit 启发式算法得到快速

的近似解。具体的解法将在第五章详细介绍。

最终，内存分配优化模块可以得到每一个内存块的合理摆放地址，使得内存块的摆放更加紧凑，提升内存的利用率。更具体的介绍在本文第五章。

3.4 METS 的局限性

本文研究 DNN 的内存优化。我们的方法基于这样的观察：对于不同的输入和可学习的参数，模型在训练的过程中以相同的方式完成计算。也即 METS 的优化依赖于模型的前期迭代与后续迭代保持访存行为一致，这一特性称为访存迭代不变性。在卷积神经网络（CNN）如 AlexNet, GoogLeNet, ResNet 和 Inception-ResNet 在内的许多网络中，确实存在这种情况。除此之外，在以 RNN、LSTM、Transformer 等模块为元素的自然语言处理模型中也存在这一情况。基于此观察，本文分析前期迭代运行中的内存使用行为，然后利用 Profile 信息查找自动内存交换和全局内存分配的策略，以最小化后续运行中的峰值内存使用情况。

因此，METS 的优化存在一定的局限性，表现在无法适用于与不具有访存行为迭代不变特性的模型训练中。对于训练过程中输入大小可变的模型，或者是网络的模型结构会在训练过程中发生改变的模型，都不能进行优化。主要原因在于，上述一类模型的不同迭代之间，内存行为会发生剧烈的变化，因而基于 profile 信息的优化策略，不能有效的应用在后续的训练迭代计算过程中。

考虑到基于反向传播的深度学习模型依然是目前最具代表性也是应用最为广泛的 AI 模型，本文的 METS 系统可以被有效应用在这些模型中，因此 METS 的应用范围依然是非常广泛的。

第 4 章 基于代价模型的自动内存交换技术

缓解 GPU 内存负载压力的一种有效方法是：在模型训练期间，在主机和 GPU 内存之间交换数据，这种方法可以在不影响准确性的情况下确保内存负载被有效降低，并且存在内存拷贝开销被计算隐藏的可能性。第二章讲到的几种技术发展趋势都证实了内存交换的方法是有吸引力的。首先，主机 CPU 的内存通常比 GPU 的内存大得多。目前最大的 GPU 内存只有 30GB，而 CPU 主存的大小通常可以达到几百 G，因此在 DNN 训练负载的有限压力之下，我们可以认为 CPU 主存是有无限大小的。此外，随着 PCIe 5.0 的到来和 GPU 提供的高速互连总线（例如，用于 Nvidia GPU 的 NVLink^[39]），主机 CPU 和 GPU 之间的通信带宽得到了充分地发挥。但是，与现有解决方案相比，由于 DNN 工作负载的独特存储行为，我们通过前期的内存行为分析可以将 DNN 训练过程中的内存分配，释放，和读写的行为进行分析。通过对 DNN 训练前期迭代中内存行为的观察和研究，系统不需要了解 DNN 的计算图结构就可以给出自动的内存调度策略，通过降低内存负载峰值的方式减轻 GPU 内存不足的压力。

4.1 DNN 内存负载行为分析

为了解决 DNN 工作负载的内存挑战，有必要对 DNN 内存负载行为进行分析，了解其特点并找到系统的设计原则。我们通过检测深度学习编程框架的内存分配器，精确追踪 GPU 每个内存块的行为，从而仔细表征深度学习工作负载的内存行为。

如图 4.1 所示为 ResNet^[40]、Vgg、GAN^[41]和 RNN^[42]四种模型的内存负载变化情况。这里的内存负载指的是在模型训练过程中，某一时刻驻留在 GPU 内的最大数据量总和。其大小通常不等于 GPU 内存的占用量，由于内存的动态分配和释放，内存碎片总是存在。从整体上看，内存占用量随着内存负载峰值的增长而增长的，要想正确运行 DNN 训练应用，GPU 就必须满足其内存负载峰值的要求，内存负载峰值是决定应用能否正常执行的关键。如果系统可以将内存负载的峰值

有效降低，意味着 GPU 不需要为更多的数据准备内存空间，最终就体现为 GPU 的内存占用量得到减少，内存得到了高效使用。因此降低内存负载峰值是自动内存交换技术的目标。

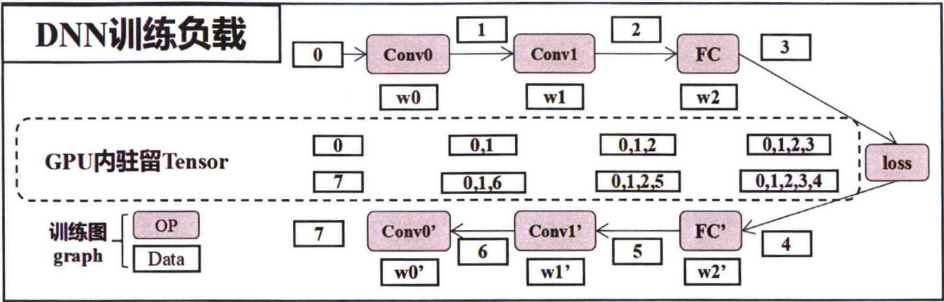


图 4.1 DNN 训练负载 GPU 数据驻留示意图

Figure4.1 Schematic diagram of DNN training GPU data residency

如图 4.1 所示，内存负载峰值通常出现在 DNN 训练的反向传播阶段，在前向传播完成的同时，也将每一层的特征图驻留在内存中。因为这些数据在反向传播的过程中还会被使用到，只有当反向传播计算完成之后，这些数据才能被释放。在基于计算图的编程框架中，通过引用计数来维护垃圾回收机制，DNN 训练的过程也被构造为一个计算流图，虽然完成了前向计算，但是对应的数据还受到反向传播算子的依赖，因此不能被释放掉。正向传播和反向传播的计算都需要特征图，只有在反向传播计算完成后，对应层的特征图占用的内存空间才能够被释放。从图 4.1 中可以看到，特征图在正向和反向计算之间存在很大的重用距离，往往第一层特征图先驻留在 GPU 内，却是最后一个被释放的，这样的特征给深度学习训练场景下的内存交换调度带来很多的机会。引出了本章的问题解决思路，即尽可能将重用距离长，且大小可观的内存块及时交换到 CPU 主存，从而有效降低内存负载峰值。

内存块重用距离和内存块大小以及动态负载信息可以通过对训练前期迭代的内存行为分析提取得到。这一设计基于 DNN 训练负载特殊的迭代性，也即初期迭代的内存负载行为与模型后期迭代完全一致。由于内存交换策略不改变模型训练计算的准确性，系统完全可以将优化后的内存交换策略应用在模型训练的后续迭代中。

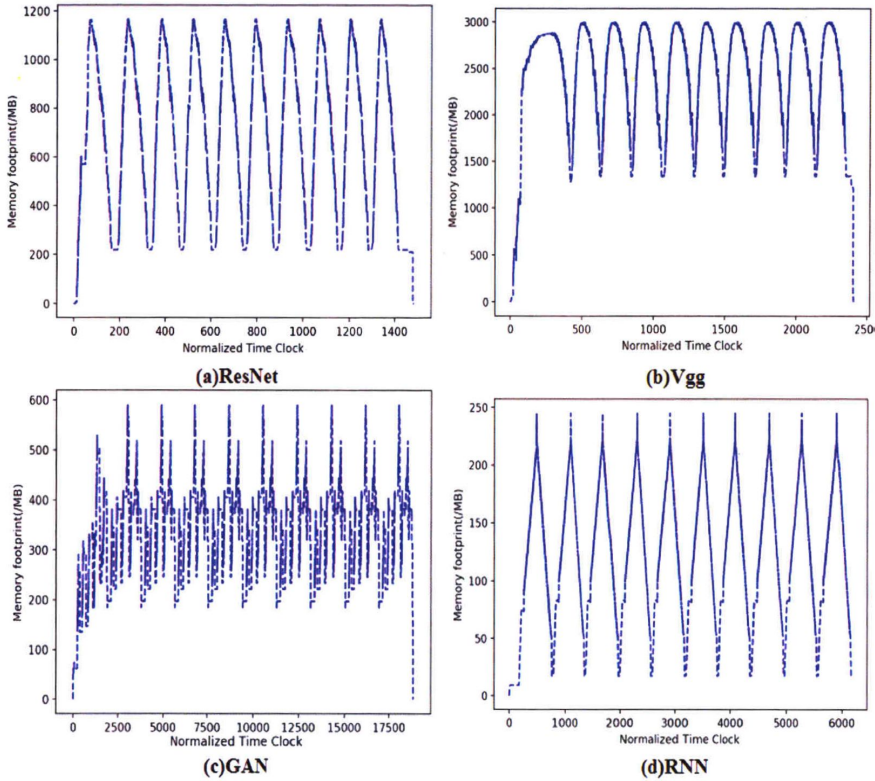


图 4.2 DNN 内存负载变化曲线

Figure 4.2 DNN memory load curve

如图 4.2 给出的是四种模型前 10 次迭代的内存负载曲线, 可以进一步证明 DNN 训练天然具备迭代的特征, 其内存访问行为比较稳定, 这为自动挖掘内存块的访存行为和访存模式提供了可能, 为了保证轻量级可以把前五轮迭代自动生成的交换策略直接应用于后续大量训练迭代。

4.2 主机和 GPU 内存交换行为分析

本节主要对主机和 GPU 之间的内存交换行为进行了分析, 在 4.2.1 中对主机和 GPU 之间内存交换中的拷贝开销进行刻画, 找到精确预测内存块拷贝延迟的数学模型, 并给出求解这一模型的方法。在 4.2.2 中以 ResNet18 训练中的内存行为为例, 分析了其中内存行为的访问间隔和内存块大小分布, 并给出了自动内存交换模块中代价模型的设计原则。

4.2.1 主机和 GPU 内存交换开销刻画

上一小节提到，为了降低 DNN 的内存负载压力，就必须将暂时不用的特征图调度到 CPU 主存，并在下一次计算需要之前拷贝回 GPU。这里涉及 CPU 和 GPU 之间的内存拷贝，其拷贝开销需要认真地研究。由于 GPU 的带宽是固定的，我们可以将其作为输入，得到将一个内存块由 CPU 拷贝到 GPU 需要花的时间，以及从 GPU 拷贝到 CPU 需要花的时间。由于我们在同一时刻只能拷贝一个内存块，这个关系可以理解为一个线性函数。

$$T = V * \text{size} + O \quad (4.1)$$

T 是 CPU 到 GPU 的内存拷贝开销，以时间为单位，单位是(ns)纳秒。 V 为 CPU 与 GPU 之间的拷贝速率，代表 1ns 能拷贝的数据量， size 为内存块的大小，单位是 byte， O 是执行内存拷贝的启动开销，通常为一个定值，单位是 ns。

为了精确的求得 CPU 和 GPU 直接的拷贝开销，我们将 CPU 到 GPU 和 GPU 到 CPU 的拷贝开销分开考虑。可以通过求解这两个线性关系来刻画 CPU 和 GPU 之间的内存交换开销：

$$T_{\text{swapin}} = V_{\text{h2d}} * S + C_{\text{h2d}} \quad (4.2)$$

$$T_{\text{swapout}} = V_{\text{d2h}} * S + C_{\text{d2h}} \quad (4.3)$$

V_{h2d} 是 CPU 拷贝到 GPU 的数据传输速率， C_{h2d} 是 CPU 拷贝到 GPU 的启动开销， V_{d2h} 是 GPU 拷贝到 CPU 的数据传输速率， C_{d2h} 是 GPU 拷贝到 CPU 的启动开销。这四个参数都是以上两个线性函数的参数，这些参数越准确，我们预测的拷贝开销就越精准，为了尽可能地得到准确的参数，我们选择使用最小二乘法来求解这两对参数。最小二乘法需要一定数量的样本，我们通过对内存负载的内存块进行聚类分析，得到了对应 DNN 应用中最常用的 20 个 Size 大小，及其实测拷贝时间，通过最小二乘法来求解得到当前 DNN 应用所适合的内存交换开销预测函数。

$$[(T_1, S_1), (T_2, S_2), \dots, (T_n, S_n)] \quad (4.4)$$

$$\begin{aligned}
\arg \min_{V,C} \sum_{i=0}^n \sigma_i^2 &= \arg \min_{V,C} \sum_{i=0}^n (T_i - T'_i)^2 \\
&= \arg \min_{V,C} \sum_{i=0}^n (T_i - V * S_i - C)^2
\end{aligned} \tag{4.5}$$

4.2.2 代价模型设计原则分析

为了得到自动内存交换模块中代价模型的设计思路，我们对 ResNet18 训练期间内存块的访问间隔进行了分析。图 4.3 为 ResNet18 训练中内存块访问间隔的 CDF（累积分布函数）图。访问时间间隔是指两次相邻访问（包括读取和写入）同一内存块之间经过的时间。如图所示，x 轴是 ResNet18 训练迭代中内存块的访问间隔，y 轴是其累积分布密度。表示访问间隔小于等于 x 的总数占总体访问间隔数量的比例。图 4.3 显示 80% 的内存行为的访问时间间隔小于 175us。我们已经通过 CUDA SDK 样本中的带宽测试工具测量了设备 GPU 内存和主机 CPU 内存之间的内存复制带宽。从主机到设备的固定内存传输带宽为 7.6GB/s，从设备到主机的固定内存传输带宽为 7.7GB/s。这意味着在不影响运行时性能的情况下，这部分内存行为的交换内存块大小最大值约为 665KB（7.6GB/s * 175us / 2）。也就是说，这 80% 的内存行为的间隔中，如果所拷贝的内存块大小大于 665kb，则内存交换带来就不能带来收益。此外，如图 4.4 的箱线图所示，大多数内存行为的访问时间间隔集中在 45us 和 225us 之间，并且它们的分布非常稳定。

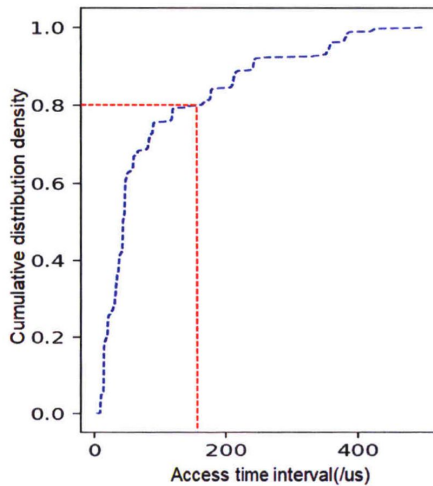


图 4.3 内存块访问间隔累积分布函数

Figure4.3 Cumulative distribution function curve of memory block access interval

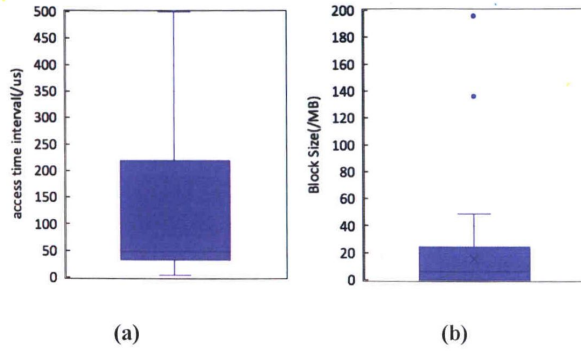


图 4.4 访问间隔(a)和内存块大小(b)的箱线图

Figure4.4 Box plot of access interval (a) and memory block size (b)

图 4.5 为 ResNet18 训练中详细的存储行为记录的部分截取，包括访问间隔以及相应的内存块大小。横轴为内存访问行为索引，纵轴为内存块大小以及内存访问间隔大小。黄色图线表征内存访问间隔，蓝色图线表征访问内存块。从图 4.5 中可以看到，大多数内存访问行为的时间间隔都比较小，在这些短间隔内进行内存交换的收益很小几乎可以忽略不计，但是，在红色箭头指向的区域，存在一些访问时间间隔大，相应的内存块大小也大的样本。

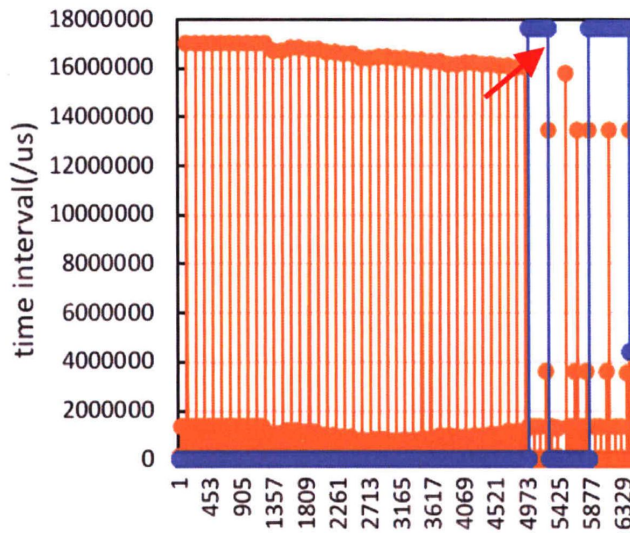


图 4.5 ResNet 训练中的访问间隔和相应的内存块大小

Figure4.5 Pair of access intervals and corresponding memory block size in ResNet training

因此基于代价模型的自动内存交换模块的设计原则即为，将访问间隔大，且内存块自身的大小可观的内存块换出到 CPU 主存，并且在下次访问之前尽可能及

时地换入到 GPU 内，达到最大限度缓解 GPU 内存负载压力并尽可能减少运行时开销的效果。因为在减少 DNN 内存负载压力上，访问间隔大，内存块也大的交换内存块是主要的贡献者。

4.3 交换内存块代价模型介绍

本节对自动内存交换模块中用于选取交换内存块的代价模型进行介绍，基于前述的设计原则，建立了两个简单的标准来过滤掉不适合被交换到 CPU 主存的内存块。首先，我们过滤出小于阈值的微小变量，例如 1 MB。由于小变量（如 DNN 中的 bias）如果被换出，对减少内存负载几乎没有影响。此外，我们必须承担数据传输中的延迟成本。其次，我们只考虑交换寿命超过高峰时间的变量。有两个优点：（i）在高峰时间附近的内存负载压力最大，因此最需要减少。（ii）如果将生命期不包含内存负载峰值的内存块交换出去，将对内存负载峰值造不成影响，降低内存负载的值没有意义，只有降低内存负载的峰值才对内存的占用起到关键的作用。换句话说，通过这两条标准，找到了有潜在收益的内存块，交换这些内存块有助于在很短的时间内有效减少内存负载。

在得到有潜在收益的内存块集合之后，代价模型负责仔细的计算各个内存块的交换收益，通过计算每一个内存块的收益进行内存块的排序，进而指导调度模块执行内存交换事件。

为评估一个内存块是否值得交换，主要关注以下几点：① 内存块的大小，② 内存块可以允许在 GPU 内存中缺失的时间跨度（从 Swapout 到 Swapin），③ 训练迭代中生命期的始末位置。为了量化候选内存块或 Tensor 的相对优先级，我们提出了内存块的交换收益：

$$G = AT = T - T_{swapin} - T_{swapout} \quad (4.6)$$

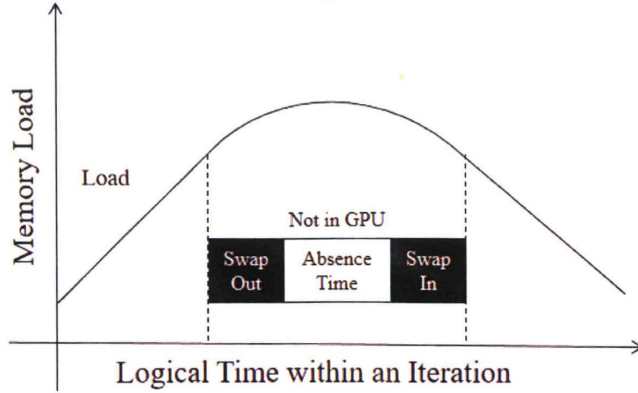


图 4.6 缺失时长示意图

Figure4.6 Schematic diagram of absence time

首先以缺失时长 AT (Absence Time) 作为内存块的交换收益, 这里 G 为 AT 收益值, T 为内存块的访问间隔时间, T_{swapin} 和 $T_{swapout}$ 为 Swapin 和 Swapout 需要消耗的时间。因此 AT 可以被视作为内存块连续两次访问之间的时间差, 减去 Swapout 和 Swapin 的时间, 所得到的就是内存块可以有多长时间不被驻留在 GPU 内存之中。但是这一收益函数只考虑了内存块的缺失时长, 却没有考虑到内存块的大小, 如果内存块很小但是缺失时长很大, 对内存负载的降低也并不能起到显著的作用, 所以这里对内存块交换的收益进行了修正提出了带权值的缺失时间:

$$G = WAT = S * (T - T_{swapin} - T_{swapout}) \quad (4.7)$$

带权值的缺失时间 WAT(Weighted Absence Time)作为内存块交换的新收益表示。它是 AT 与内存块大小 S 的乘积, 它量化了一段时间内减少的内存负载量。系统可以优先选择具有较大 WAT 的内存块进行交换, 因为它会减少内存的较大部分或者更长的时间。也可以同时选到较大的内存块和较长的缺失区间, 因此相较于 AT, WAT 更加适合于降低内存负载。但是 WAT 并没有考虑到内存负载峰值的变化, 当我们将部分内存块交换出去之后, 内存负载的峰值就会发生改变。由于内存交换的目的是降低内存负载的峰值, 所以在做出一次内存拷贝决策指挥, 需要重新计算内存负载的峰值, 以动态执行的形式计算每一个内存块的交换收益。具体来说, 就是必须在每一次选择待交换的内存块之前, 明确内存负载的峰值, 从而找到在最新内存负载峰值下, 计算去除当前内存块可以给内存负载峰值的下

降带来多少收益。这种方法在本文中被称作动态权重缺失时间 DWAT (Dynamic Weighted Absence Time) 其计算方法如 Algorithm1 所示:

Algorithm 1: get DWAT score

算法输入: 1.存储访问行为序列 2.用户指定的最大内存负载上限

算法输出: 满足用户指定最大内存上限的内存交换计划

算法步骤

```

1: Function schedule(vector<R> mem_request){
2:   Max_mem_load = get_peak_mem_load(vector<R> mem_request)
3:   //得到负载峰值
4:   while(mem_load_limit<=Max_mem_load){
5:     //选择 Swapblock, 添加内存交换事件
6:     for Each (A, B) in vector<R>; do
7:       If A==Malloc
8:         mem_load = mem_load + B.size
9:       If A==Free
10:        mem_load = mem_load - B.size
11:      Calculate memory saving gain G WAT for B;
12:      //计算当前负载峰值下 WAT
13:      G=G/mem_load
14:      Sort G and determine candidate swapping memory blocks;
15:      //排序选取最大 WAT
16:      Update vector<R>//添加内存交换事件, 更新存储访问行为序列
17:      Max_mem_load = get_peak_mem_load(vector<R>)//更新负载峰值
18:    }
19:    Update meta_sched;//调度内存交换事件, 记录同步事件&流
20:    Construct memory pool;//构建内存池
21:    PerformSwapping(pool, meta_sched);//执行 Swap 操作
22:    Return 0;
23:  }
```

用户需要输入一个内存负载的上限, 只要当前内存负载峰值高于这个上限, 就必须去寻找下一个需要被交换的内存块及其交换区间。寻找的方法是通过计算每一个内存块的 WAT 交换收益, 并且选择收益最大的内存块及区间插入内存交换操作。内存交换操作的插入包括换入和换出两种, 换入操作先在 GPU 内存

中分配好指定大小的空间，在执行内存拷贝将数据写在指定的地址上；换出操作先要在 CPU 主存上分配好指定大小的空间，待到 GPU 内存完全拷贝给 CPU 主存之时，才释放掉 GPU 上的内存空间。

重复这样的执行过程，直到可以满足最高内存负载上限时，不再增加内存交换事件，最终系统得到了一个待执行的内存交换事件表，其中包括了每一个需要被交换的内存块及其交换区间。

4.4 交换事件调度设计

内存交换在不使用所选内存块时将其交换出来，并在下一次访问之前换入它们。为了最大限度地提高交换和同步重叠率，我们通过调用 GPU 供应商提供的异步执行 API（例如，针对 Nvidia GPU 的 `cudaStream_t`），使用单独的流以异步方式将选定的存储块传入/传出外部存储器。我们按访问时间的升序安排所有交换事件。这些异步交换流负责数据交换，分配和释放。为了确保正确性，调度程序的工作方式是，只有在上一个交换入/出事件完成后，才可以开始换入/输出存储块。换入事件是根据每个内存块的下一次访问时间进行安排的。为了确保第二次访问不会因交换事件而延迟，需要预先换入存储块，以确保其第二次访问不会由于通信而延迟。交换事件调度的设计方式使得内存块仅在上一个交换事件完成后才开始下一次交换。对于那些换出事件，它们也按时间顺序进行安排。访问完成后，可以立即换出存储块。

系统为这些选定的交换块生成一个调度表。调度表是所选交换块及其交换事件的映射。Key 是块索引，Value 是与交换事件有关的元信息。对于每个交换块，记录以下元信息：原始设备内存地址，主机 CPU 内存地址，块的大小，异步交换输入/输出事件和异步交换输入/输出流。此外，用于交换和交换动作的交换和同步事件的标志也是必需的。这些标志表示何时执行交换事件以及何时执行同步事件。然后，系统将构造内存池并记录这些选定交换块的元信息。

执行交换时，系统从调度表中查询交换标志。如果交换标志为换入（Swapin），则系统将从内存池分配内存块，并将数据从主机 CPU 异步复制到设备加速器。同时，系统必须更新此块的信息，并记录此异步事件以进行交换和同步的重叠。换

出的情况与此类似。

如果在宽松的内存负载限制下,通信和同步可以完全隐藏在计算之下。但是,在严格的内存负载限制条件下,由于无法快速换出变量,因此内存负载将超过限制。如果它在正向传播期间发生,则下一个 Malloc 将被延迟,直到交换出足够的变量并释放内存为止;否则,下一个 Malloc 将被延迟。如果在向后传播期间,仅在释放一些变量后才能开始交换。这是发生通信和同步开销并且无法隐藏的地方。

执行交换时,系统从调度表中查询交换标志。如果交换标志为换入(Swapin),则系统将从内存池分配内存块,并将数据从主机 CPU 异步复制到设备加速器。同时,系统必须更新此块的信息,并记录此异步事件以进行交换和同步的重叠。换出的情况与此类似。

4.5 工程实现介绍

```
class SwapGPU {
    Block* Malloc(size_t);
    void Free(Block*);
    void Exec(Function, vector<Block*>);
    SwapGPUPool Pool();
    map<int, Event> SwapOpt();
}
```

图 4.7 工程实现抽象

Figure 4.7 Engineering abstraction

内存交换核心功能的抽象如图 4.7 所示,我们在开源深度学习框架中添加了新的设备,其包含了基本的 Malloc 和 Free 操作,以及内存池和内存交换调度表。对每一个内存块对应的交换事件进行维护。

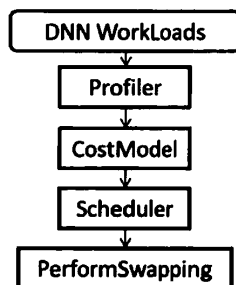


图 4.8 基于代价函数的内存交换算法流程

Figure 4.8 The execution process of memory swap algorithm based on cost model

具体的执行流程如图 4.8 所示，深度学习应用算法的训练作为工作的负载，在训练的前五个迭代中，我们先通过内存行为分析模块(Profiler)对执行过程中的访存行为进行记录，并掌握一次迭代过程中的稳定访存行为和访存模式。基于每一个内存块的访存间隔和内存块的大小，交换内存块代价模型(CostModel)可以在给定内存负载限制下对交换内存块和内存交换区间进行决策。最后交换事件调度器(scheduler)对前面构建好的内存交换事件进行统一调度，按照访问时间的升序安排交换事件，为了确保交换事件准确无误，必须在完成上一次交换事件之后才能发起下一次交换事件。

4.6 内存交换策略对比分析

为了证明本文提出的自动内存交换技术的创新性，本节对现有学术界最新有代表性的内存优化解决方案进行对比分析，尤其是对这些解决方案中的内存交换策略进行分析。以便于找到本章基于代价模型的自动内存交换技术在内存交换策略上与现有内存交换策略的不同之处。

如表 4.1 所示，这里对目前已有的 DNN 内存降低解决方案中的内存交换策略进行了对比。vDNN 选择交换所有的层或者是之间交换卷积层，不加分辨的策略对于复杂多变的深度学习应用来说并不适合。SuperNeurons 选择只交换那些卷积层，其考虑是卷积层的计算时间总是大于内存拷贝的时间，这样内存拷贝的开销可以被计算很好地隐藏，这样的做法没有考虑到除卷积层之外其他计算层中计算对内存拷贝开销的隐藏，同时由于只对卷积层进行交换，SuperNeurons 的内存消耗还有进一步降低的空间。Capuchin 使用基于缺失时长(absence time)的代价模型来选择 Swaptensor，其优点在于可以全自动地对 DNN 训练中的张量进行交换，避免了人工制定内存交换策略带来不灵活应对上层应用变化的问题。但是基于缺失时长的代价模型没有很好地利用张量的大小，容易将小内存块作为交换内存块，这样的策略对于降低内存负载峰值的作用有限。GeePS 主要借助用户自定义来确定内存交换的层，这样的方法需要用户对内存占用和各个算子的计算时长有深入地了解才能制定出合理的交换策略。TF-LMS 提出了一些形式化的图形重写规则，

以便在计算图上插入换入或换出节点，其对上层应用计算图有依赖，面对新应用和新算子不能灵活应对。SwapAdvisor 基于 DNN 工作负载的计算图，按照三个维度执行联合优化，包括 Op 调度，内存分配和交换张量决策。这种方法将内存分配和算子调度与内存交换策略结合起来优化，存在很大的优化空间，在有限复杂度下搜索这一空间最优解存在较大困难。

表 4.1 现有解决方案内存交换策略对比

Table 4.1 Comparison of memory swap strategies for existing solutions

解决方案	内存交换策略
vDNN[30]	交换所有层
SuperNeurons[32]	只交换卷积层
Capuchin[36]	基于缺失时长 代价模型选择 SwapTensor
GeePS[29]	用户自定义
TF-LMS[33]	Graph 重写规则
SwapAdvisor[34]	Op 调度，内存分配和交换张量决策联合优化

综上所述，本章的自动内存交换模块以 Tensor 为粒度，通过代价模型预测训练全程中内存块的交换收益，帮助系统自动挑选交换内存块以及交换时间区间。一方面摆脱了人工指定内存交换策略带来的不灵活；另一方面，由于对于计算图不感知，更加便于系统应对复杂多变的上层应用。相比于简单基于缺失时长的代价模型，本章的自动内存交换模块设计了动态带权值缺失时长(DWAT)的代价模型来权衡内存块大小和内存块缺失时长对降低内存负载峰值的贡献，更加直接有效地降低内存负载峰值并尽可能地减少了运行时的开销。

4.7 本章总结

本章提出了基于代价模型的内存交换技术来有效降低内存负载峰值，提升内存使用效率。4.1 节对 DNN 训练负载的内存行为进行分析，找到自动内存交换模块的设计思路和出发点，并提出借助 DNN 训练迭代特性，由前期迭代中内存行为模式的分析结果自动生成内存交换计划的策略。4.2 节对主机和 GPU 之间的内存交换开销进行了刻画，并给出了交换内存块代价模型的设计原则。4.3 节详细介绍

绍了交换内存块代价模型。4.4 节对内存交换事件的调度进行介绍。4.5 节从工程实现的角度对自动内存交换模块进行程序抽象。最后，在 4.6 节将本章提出的内存交换策略与当前学术界的代表性成果进行比较分析，证明了本章提出方法的创新性。

第 5 章 基于全局访存行为的内存分配优化技术

第四章讲到自动内存交换通过降低内存负载峰值来缓解内存，内存负载的峰值并不等于实际内存占用值，主要是因为内存碎片的存在。目前的深度学习编程框架为了适应各类应用的需求，采用了动态内存池的办法来处理内存分配和释放请求，这样的方式无法预知未来的内存请求，因此不可避免地带来了内存碎片。内存碎片的存在降低了 GPU 内存利用率，本章通过开发一种新颖的基于全局访存模式引导的内存分配优化算法来解决此难题，以在 DNN 训练期间高效，快速地分配内存块。

本章研究的主要内容是 DNN 训练的内存分配优化，致力于解决现有动态内存池的内存碎片问题。基于前文的观察，DNN 训练任务，对于不同大小的输入和可学习的参数，都会以相同的方式计算网络模型的传播；我们称这样的特性为迭代性，即网络训练的内存行为在每一次迭代过程中都是相同的。在许多网络中都遵循这一特性，包括卷积神经网络（CNN），例如 AlexNet, GoogLeNet, ResNet 和 Inception-ResNet，以及 LSTM^[43]，BERT^[44]和 Seq2Seq^[45]等模型。基于此观察，可以分析样本运行中的内存使用情况作为内存行为模式(Profile)，然后利用该 Profile 文件查找内存分配的方案，以最小化后续运行中的峰值内存使用情况。

我们将内存分配问题抽象为二维矩形包装问题的一种特殊情况，已知该问题是 NP 难。本章开发了一种简单快速的启发式方法，可以快速生成良好的近似解。在以前的工作中，一个变量仅与另一个变量（一对一）共享内存。通过全局内存分配优化，这些块之间可以共享内存，而不会导致生命周期重叠，并且这种共享不限于一对一共享，同时也允许更高级的共享，以便一个大块可以与几个小块共享内存，反之亦然。

5.1 动态内存池碎片问题剖析

Global Memory 是 GPU 上空间最大的存储介质，对此 CUDA 提供了 `cudaMalloc` 和 `cudaFree` 来申请和释放该区域存储，这样做的缺点在于：由于所申

请内存块的大小不定，当频繁使用时会造成大量的内存碎片并进而降低性能。实际上这两个 API 的使用开销还是比较大的，业界普遍采用框架内存池进行管理，既采用二次分配的方案，而该方案又可以分为动态和静态两种思路。静态分配是依据计算图的结构以及算子之间的数据依赖关系提前进行分析，编排分配，训练过程中没有实际的物理或逻辑上的显存申请和释放发生；动态分配是采用内存池的方式，类似操作系统对于物理内存的管理机制，好处是不用提前根据模型进行编排，使用更加灵活，例如在动态图的场景下没有办法针对计算图进行提前的编排，但是在训练过程中依然需要显示执行逻辑上的申请和释放操作，依然会有轻微的开销。

内存池技术即为动态分配内存管理技术，内存池是在真正使用内存之前，先申请分配一定数量的、大小相等(一般情况下)的内存块留作备用。当有新的内存需求时，就从内存池中分出一部分内存块，若内存块不够再继续申请新的内存。这样做的一个显著优点是，使得内存分配效率得到提升。动态分配大多基于 Best-Fit-with-Coalecing^[46]的策略，通过维护 used-blocks 和 unused-blocks 两个链表来管理内存池中的所有内存块。Best-Fit 是指每次为 DNN 训练负载分配内存时，总是把能满足大小要求且最小的 unused-block 分配给内存请求，避免大材小用。为了加速寻找，该算法要求将所有的空闲分区按其容量以从小到大的顺序形成一个 unused block 链。这样，第一次找到的能满足要求的空闲区，必然是最佳的。单纯看这一逻辑似乎是最优的，然而在训练迭代的整体上看却不一定。因为每次分配后所切割下来的剩余部分总是最小的，这样，在存储器中会留下许多难以利用的未使用区域，这些小的未使用空间难以被未来的内存请求所使用，形成了内存碎片。

如图 5.1 所示，动态内存分配机制无法预知未来的内存请求时间以及请求内存块的大小，在应对新的内存请求时总是按照 Best-Fit 的策略寻找满足请求大小的最小未使用内存块。在程序动态执行的过程中，随着内存分配和释放的进行，内存池中容易出现小的未使用内存块，这些未使用内存块只有在地址相邻时才可以被合并为更大的未使用内存块。因此当小的未使用内存块无法被后续内存请求所利用时，容易出现内存碎片。

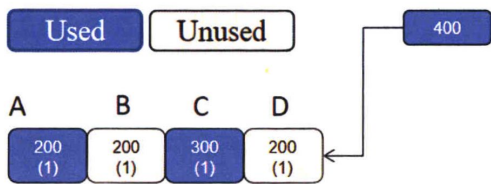


图 5.1 内存碎片产生示意图

Figure5.1 Schematic diagram of memory fragmentation

如图所示，蓝色的内存块为已使用内存块，灰色的内存块为未使用内存块。以内存块 A 为例，其大小为 200MB，处于已使用状态，当前内存池收到 400MB 的内存请求时，虽然内存池内未使用内存块的大小总量可以满足 400MB 的需求，但是由于两个未使用的内存块在地址上不相邻，则无法被合并作为统一的内存块来满足 400MB 的请求。内存池在未使用内存块中搜索满足需要的内存块，如果找不到满足需要的内存块，则需要通过 cuMalloc 等 API 向 GPU 申请新的物理内存加入内存池中。这就带来了内存占用量的增加，使得深度学习训练系统的内存使用率降低。

我们已经在 ResNet 工作负载训练中绘制了前十轮迭代的甘特图。图 1.6 中的 x 轴是归一化的时间戳，y 轴是 GPU 的归一化内存地址。图中的每个矩形代表对 GPU 的设备内存中某个内存块的内存访问行为。矩形的宽度是从内存块被分配到被释放之间的所经过的时间，可以视为当前内存块的寿命。矩形的高度表示当前内存块的大小。显然，我们可以从图中看到，在 ResNet 的训练过程中，前十轮迭代中存在重复的内存访问模式。此外，沿 y 轴的矩形之间的重叠表示不同内存块的有在有效范围内的复用。沿 y 轴的两个矩形之间的空白表示 GPU 的内存碎片。从图 1.6 中可以看到，ResNet 的训练过程中，还是存在相当数量的内存碎片。

综上所述，现有的动态内存池分配器由于没办法预知全局的内存请求，所以不可避免地导致内存碎片的产生，且在复杂的 DNN 模型训练中内存碎片数量可观。

5.2 内存分配优化问题抽象

内存分配通常可以视为**二维条带打包问题（2SP two dimensional strip packing problem）**。它要求将一组矩形物品放置在具有固定宽度的容器中，并使可变

高度最小化。一个存储块对应于一个矩形项目，其分配时间为宽度，其存储大小为高度。在本文中，我们处理一个特殊情况，其中所有存储块的分配时间都固定为输入。该问题也称为动态存储分配问题（DSA），这是一个典型的 NP-hard 问题^[49]。

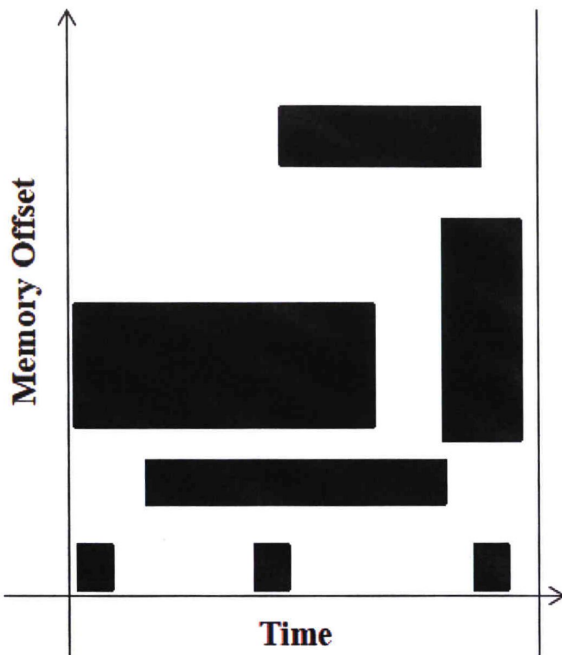


图 5.2 二维条带打包问题示意图

Figure 5.2 Schematic diagram of two-dimensional strip packing problem

内存的分配问题可以通过整数线性规划的方法求解其最优解，但是基于整数线性规划的方法所需要的优化求解时间巨大，因此我们开发了一种简单快速的启发式方法，可以快速生成良好的近似解。运行示例算法后，我们的方法为每个内存请求生成一个绝对内存地址，并在后续运行中将其返回。如前文所述，深度学习模型在训练过程中具备很强的迭代性，有着稳定的访存模式。我们可以分析样本运行中的内存使用情况，然后利用该配置文件查找内存分配的策略，以最小化后续运行中的峰值内存使用情况。因此，与动态内存分配相比，由于我们的方法可在优化内存分配的过程中掌握全局的访存信息，可以做到在全局范围内对内存的摆放进行优化，相比较于动态内存分配，我们的方法内存碎片更小，运行时成本更低。如图 5.2 所示，由于我们掌握了一次迭代过程中的完整访存行为，那么内存的分配和摆放变成了一个确定的二维条带包装问题，只要我们求解这一问

题，就可以使得内存的摆放更加紧凑，内存的碎片极大地降低。

本章提出了面向深度学习训练的基于全局访存行为的内存摆放优化技术。我们的方法在稳定迭代的访存模式下，可以更加优化内存的使用，并且一旦优化了内存使用，就永远不会产生性能开销。针对 NP 难的二维条带打包问题，本章展示了 DSA 的混合整数型线性规划(MILP Mix Interger Linear Programming)解法，并且提出了一种启发式算法来求解大型实例。

5.3 MILP 求解方法介绍

混合整数线性规划解法是求解优化问题的常用方法，通过合理地设计约束条件，借助成熟的优化库可以很方便地对问题进行优化。我们首先介绍内存分配优化的参数。假设置文件给出了请求的内存块的数量，请求和释放存储块的时间以及存储块的大小。并且将可用的最大内存大小作为参数。

$n \in \mathbb{Z}$ ：内存块的总数。

$B = \{1, \dots, n\}$ ：内存块 id 的集合。

$W \in \mathbb{N}$ ：可用的最大内存空间。

$w_i \in \mathbb{N}(i \in B)$ ：内存块 i 的大小。

$\underline{y}_i \in \mathbb{N}(i \in B)$ ：内存块 i 的分配时间戳。

$\overline{y}_i \in \mathbb{N}(i \in B)$ ：内存块 i 的释放时间戳。

假设这些参数在深度学习模型的整个训练过程中不会改变。在时间段 $[\underline{y}_i, \overline{y}_i)$ 中分配了一个内存块 i ；那么这一区间就是内存块 i 的生命周期。

线性规划的决策变量包括：

$u \in \mathbb{Z}$ ：最大内存使用量。

$x_i \in \mathbb{Z}(i \in B)$ ：内存块 i 的内存偏移，或其起始地址与内存池起始地址的偏移量。

$z_{ij} \in \{0, 1\}((i, j) \in E)$ ：0 代表内存块 i 的分配起始地址比内存块 j 的分配起始地址低(i.e., $x_i + w_i \leq x_j$)。1 代表内存块 j 的分配起始地址比内存块 i 的分配起始

地址低 (i.e., $x_j + w_j \leq x_i$)。这里可以将内存块 i 的区间 $[x_i, x_i + w_i)$ 看作其存储地址区间。

这里将内存偏移量分配给内存块，以便在任何给定时间没有两个内存块占用相同的地址空间。我们不需要遍历所有的内存块对来检查这一冲突，而是只需要检查具有重叠生命周期的内存块就足够了。为此，我们选择构建内存块生命周期的冲突图。

$$E = \{(i, j) \in B \times B \mid i < j \text{ and } [\underline{y}_i, \overline{y}_i) \cap [\underline{y}_j, \overline{y}_j) \neq \emptyset\} \quad (5.1)$$

这是一组具有重叠生命周期的内存块对。请注意，不在 E 中的任何两个内存块不会同时共享相同的地址空间，因为它们的生存期不重叠。我们做内存分配优化的最终目的是最大限度地减少峰值内存使用量。因此有了决策变量，约束条件以及必要的常数数量之后，我们以混合整数规划 MILP^[51] (Mixed-integer linear programming) 的形式给出优化的目标和必须遵守的约束条件，如下所示：

$$\min u \quad (5.2)$$

$$s.t. \quad x_i + w_i \leq u \quad (5.3)$$

$$x_i + w_i \leq x_j + z_{ij}W, (i, j) \in E \quad (5.4)$$

$$x_j + w_j \leq x_i + (1 - z_{ij})W, (i, j) \in W \quad (5.5)$$

$$0 \leq u \leq W \quad (5.6)$$

通过这样的设定，系统可以借助 CPLEX 求解这一混合整数线性规划问题，小规模的问题比如 MLP, LeNet, CifarNet^[49]等网络可以在一小时的运行时间内得到最优解，由于这里涉及冲突图的构建，因此约束条件的数量容易受到冲突图的规模限制，约束条件的数量与内存块数量的平方成正比。因此当一个模型的训练过程中内存块数量越大，带来的约束条件数量也就越大。

MILP 的求解方法虽然可以得到问题的最优解，但是在执行效率上非常不理想，在深度学习训练的场景下不能承受数个小时的优化时间消耗。因此必须找到启发式算法来快速求得足够好的近似解以适应 DNN 训练的实际要求。

5.4 Best-Fit 启发式算法求解介绍

深度学习模型训练中涉及的内存块如果过多，那么内存块之间的依赖关系就会达到数万个，这时候整数线性规划的方法就不能顺利地解决大规模的约束条件下的求解。我们需要一个快速的解法来找到优化后的近似解。

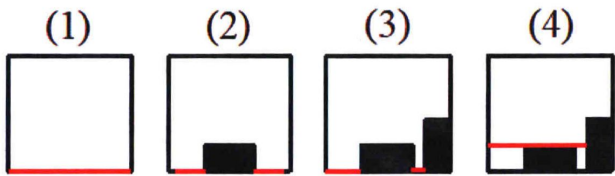


图 5.3 Best-Fit 启发式算法示意图

Figure5.3 Schematic diagram of Best-Fit heuristic algorithm

算法的目标是将所有内存块都放置在矩形中，以便将顶部内存块放置在尽可能低的位置。分配算法如图 5.3 所示，重复两种操作，直到放置了所有内存块：
①选择一个地址偏移；②搜索可以以所选偏移量放置，且不会与已经放置的内存块发生冲突的内存块。我们通过图中所示的示例说明启发式算法的工作方式。在启发式算法的开始（图 5.3(1)）中，由于未放置任何内存块，因此我们选择零作为内存偏移量。在搜索内存块时，在可以放置在当前偏移量的块中，我们总是选择生命周期最长的块。在图中的情况下，我们选择寿命最长的内存块并将其放置（5.3(2)）。放置后，将当前偏移量加上内存块的大小加入偏移量集合中。当存在多个偏移时，我们总是选择最低的偏移（如果存在多个最低偏移，则选择最左边的偏移）。接下来，我们为选定的新偏移量搜索一个存储块并将其放置（图 5.3(3)）。如果没有块可以放置在选定的偏移量处，我们可以将当前偏移量去除，并且更新偏移量，如图 5.3(4)所示并再次搜索内存块来放置。启发式算法的计算复杂度是 $O(n^2)$ ， n 是内存块的数量。

具体的伪代码如算法 2 所示：

Algorithm 2: Best-Fit Heuristic

算法输入：1.未放置的内存块列表；2.已放置的内存块列表；3.内存池起始地址

算法输出：每一个内存块相对于内存池起始地址的偏移量

```
1: Bool canPlaced(Block * blk,int offset){
2:    //判断当前内存块是否可以在给定 offset 下放置
```

```

3:   canPlaced=True
4:   for placed_block in placed_blocks do//遍历已放置内存块
5:       if placed_block->address_r <= offset
6:           continue
7:       if blk->Malloc_time >= placed_block->Free_time or\
8:           blk->Free_time < placed_block->Malloc_time:
9:           continue
10:      else
11:          canPlaced=False//已放置内存块与当前内存块生命期冲突，不能放置
12:  return canPlaced
13: }
14: // vector<Block*> unallocated_blocks 未放置内存块数据结构
15: // vector<Block*> allocated_blocks 已放置内存块数据结构
16: // start_address mem pool start address
17: // set offsets={0} 初始化 Offset 数据结构
18: while(unallocated_blocks.size()>0){//进行内存块放置，直到未放置列表为空
19:     offset=min(offsets);//每次总是寻找最小的 offset
20:     sorted_blocks = Sort_by_lifetime(unallocated_blocks);
21:     //根据生命期长度对未放置内存块进行排序，优先放置长周期内存块
22:     placheres=True;
23:     count=0;
24:     for block in sorted_blocks do
25:         placheres = canPlaced(block);
26:         if (placheres==True)
27:             offsets.push(block.size+offset);
28:             block.address=start_address+offset;
29:             unallocated_blocks.remove(block);
30:             allocated_blocks.push(block);
31:         else
32:             count++;
33:             continue;
34:         if (count==sorted_blocks.size())
35:             offsets.remove(offset);
36:     }

```

5.5 内存分配优化模块工程实现

我们在开源深度学习框架 TensorFlow 中整合了 Best-Fit 启发式算法，以优化 GPU 内存使用率。MIP 解法则是通过调用 CPLEX 求解库来完成。

5.5.1 全局访存行为模式提取

由于 TensorFlow 在运行时分配内存块，因此我们通过监控算法运行中的内存分配和释放操作来分析 GPU 内存使用情况。为了获得内存请求时间 y_i 和释放时间 $\bar{y}_i$ ，我们通过对 memory allocator 进行插桩来定位每一个内存块的申请和释放时间戳，借助实际运行时检测到的生命周期来生成冲突图。对于每一个申请的内存块，我们都赋给一个全局变量 λ ，该变量每完成一个内存块的处理就加一，并维护一个 key 为 λ ，value 为内存块元信息的哈希表。当收到内存大小为 s 的请求时，我们用 λ 扩展 B (内存块 ID 集合)，将 s 设置为 w_λ ，并将当前时间戳赋给 y_i 。当释放内存块 i 时， w_i 仍为 s 保持不变，而当前的时间戳会被赋给 $\bar{y}_i$ ，直到全部的内存分配和释放请求都以处理完。到此我们就得到了深度学习模型训练的 Profile 信息，后续将会借助这些信息计算内存分配优化方案，生成内存分配计划来指导后续迭代中的内存块摆放。一旦内存分配计划生成，则后续迭代中的每一个内存块的分配位置即为一个确定的绝对地址，相比于动态内存池分配的方法，我们的方法不需要在后续迭代中从内存块链表中查找不小于请求大小的最小未使用内存块。

5.5.2 内存分配执行流程

获取模型训练一次迭代的 Profile 信息之后，我们通过求解二维条带打包问题来计算一次迭代中的峰值内存使用量 u 以及每一个内存块相对于内存池的内存偏移 x_i ，然后向系统申请分配大小为 u 的 GPU 内存池；我们将内存池地起始地址记为 p 。在模型训练的剩余迭代中，对于内存块 i ，返回其在内存池中的绝对地址为 $p + x_i$ ，以请求内存块 i 。我们通过维护全局变量 λ 来识别内存块，全局变量 λ 在开始每一次迭代前向传播之前用 1 来将其初始化。当请求内存块时，我们返

回地址 $p + x_i$ 并增加 λ 。由于 DNN 训练过程中访存行为迭代不变，训练中每一次迭代的计算方法与全局访存行为模式(Profile)中的计算方法相同，在每次分配后，全局变量 λ 始终会增加。

5.5.3 单 GPU 不可训练模型解决方案

人们可能会担心，本章提出的方法在单 GPU 内存容量无法满足模型一次迭代需求时失效。因为提取 Profile 信息所执行的迭代为可能会导致内存效率低下，并且需要比物理容量需要更多的内存。即使在这种情况下，我们也可以通过使用片外内存利用来获得 Profile 信息。比如我们可以使用内存交换技术或者是 NVIDIA CUDA 中的 Unified Memory 引擎，这样的技术手段都可以使我们使用超出物理内存容量的内存来运行需要的模型，由于我们只需要执行一次迭代就可以得到 Profile 信息，因此利用片外内存空间带来的额外的性能开销下可以忽略不计，在后续迭代中可以通过禁用这些技术来避免额外性能开销的产生。由于 Profile 信息主要被利用到的是内存块的大小及其生命周期，其生命周期的绝对时间区间不是我们需要的，我们需要的仅仅是一次迭代中各个内存块之间的冲突关系，因此使用片外内存扩展 GPU 内存容量产生的 Profile 信息不会对后续优化的结果产生影响。

5.6 动态卷积工作空间分配优化

5.6.1 卷积工作空间介绍

CNN 是计算机视觉中通用的特征提取 Backbone。深度学习模型的训练中含有 CNN 结构的模型也成了模型训练的主力军，而在 CNN 模型中存在大量堆叠的卷积计算层，这些层的计算占据了 CNN 网络计算的主体。在模型训练中，非常注重模型训练的吞吐量，那么卷积算子的计算速度就至关重要。CONV 层的速度显著影响训练的速度，因为它占总计算时间的 50% 以上（图 5.4）。

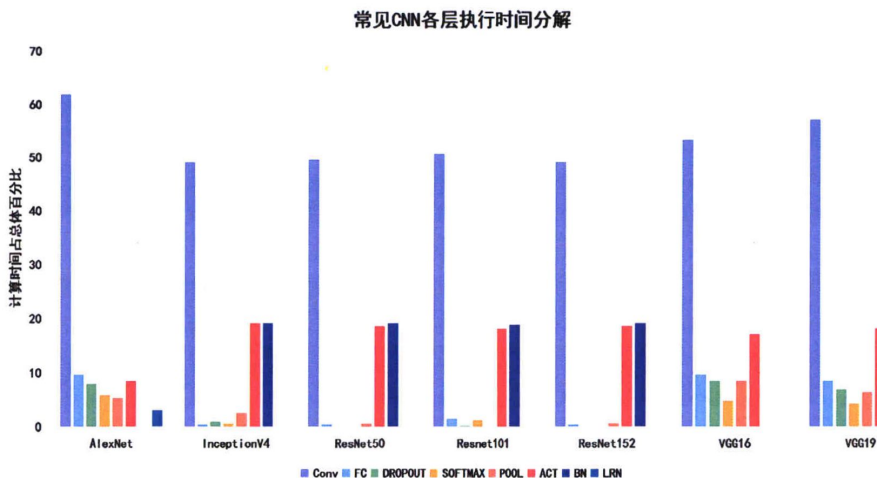


图 5.4 常见 CNN 各层执行时间分解

Figure 5.4 Common CNN execution time decomposition of layer

cuDNN 提供了几种卷积算法，例如针对不同的环境可以选择使用 FFT^[50]，Winograd^[52]和 GEMM^[53]。其中一些，特别是 FFT，需要临时卷积工作区以提供最大卷积计算速度。例如，使用内存效率最高的卷积算法（例如，cuDNN 中的隐式 GEMM）的 CONV 层需要三个数据结构，即输入/输出特征图（X 和 Y）以及层的权重（W）进行正向传播。然而，采用基于快速傅里叶变换（FFT）的算法需要一个额外的工作（WS）缓冲区来管理变换后的特征图。因此，内存也是进行高性能训练的关键因素。这里做动态卷积工作空间分配旨在结合前述内存调度算法和全局内存分配优化算法来优化 CNN 中卷积算子的计算速度，同时也加快了 CNN 整体的训练速度。如果想要得到更快的计算速度就必须牺牲内存空间作为卷积算子的临时工作空间。

5.6.2 卷积工作空间动态分配机制

通过对 Profile 信息的分析，我们可以知道，在卷积神经网络训练过程中，卷积算子工作空间具有临时申请，用完即被释放的特点，所以其生命周期往往较短，更加适合于动态申请和释放。由于生命周期较短，通过内存交换来降低卷积工作空间的占用作用非常有限。在静态全局内存分配的场景下，由于卷积算子工作空间通常较小，并且生命周期短，因此其所占据的内存空间难以被其他内存块所复用。如果卷积工作空间相关的内存块也参与全局的内存分配优化容易带来内存碎

片的增加。此外，由于 GPU 在同一时间内只能处理一个卷积层，所以卷积的工作空间不会同时存在，在任意时刻，GPU 内驻留的卷积工作空间内存块的数量不大于 1。存在卷积工作空间之间相互复用的机会。

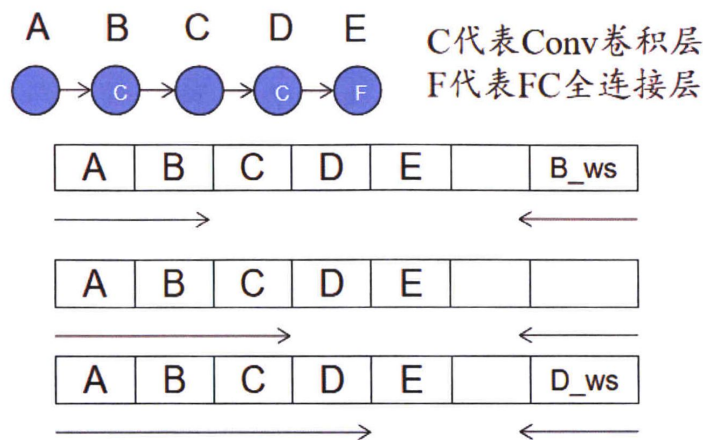


图 5.5 卷积工作空间动态分配策略

Figure5.5 Convolutional Workspace Dynamic Allocation Strategy

在这里，系统内存池将静态内存分配和动态内存分配相结合，通过前述的全局分配优化后的功能张量数据（例如特征图和模型参数）我们成为静态分配内存，而卷积算子的工作空间我们称为动态分配内存。如图 5.5 所示是一个简易的卷积神经网络，A，B，C，D，E 为神经网络的五个层。可以看到 B 和 D 为两个卷积层，左边是低地址端，右边为高地址端。当模型执行到 B 时，系统在高地址端为 B 分配了工作空间。当模型执行到 C 时，系统将 B 层计算需要的卷积工作空间释放。当模型执行到 D 时，系统为卷积层 C 在高地址端分配了工作空间。

静态分配内存从低地址到高地址进行分配，而卷积的工作空间由高地址到低地址分配，由于这些内存块分配后首先被释放的特点，在内存池的高地址段将始终只有一个内存块存在。通过双端内存分配，我们实现了静态分配与动态分配相结合。并进一步降低了内存碎片，提高了内存利用率，具体的实验验证见第六章。

5.7 本章小结

本章提出了 DNN 训练场景下的基于全局访存行为的内存分配优化算法。将内存分配优化问题抽象为二维条带打包问题，提出了混合整数线性规划的方法来求

解小规模问题的最优解，以及开发了一种简单的启发式算法，以获取高效，快速的内存分配，并将启发式算法整合到开源的深度学习编程框架中。除此之外，本章提出了静态内存分配和动态内存分配相结合的方式，进一步提高了卷积工作空间的利用率，降低了全局内存碎片的数量。

除了内存占用的降低之外，DNN 的训练速度也得到了加快。因为在实际运行示例时，系统为每个内存请求生成一个绝对内存地址，并在后续运行中将其返回。因此，与许多深度学习框架中的标准方法动态内存分配相比，我们的内存分配在后续运行中的运行时成本非常低。此外，由于静态内存分配优化算法，带来的内存减少允许用户使用更大的 BatchSize 进行训练，导致 GPU 计算资源的利用率提高并进一步加快训练速度。相关的实验测评在第六章进行详细介绍。

综上所述，本章提出的方法主要贡献如下：

首先，借助了 DNN 训练中多次迭代不变的性质，通过利用训练中单次迭代的全局内存行为信息，对内存块的分配位置进行优化，并将优化结果应用在 DNN 训练的后续迭代。将动态内存分配转变为静态的内存分配，系统为单次迭代中每一个内存块分配一个绝对地址，在后续迭代中省去了动态内存池中对内存块链表的频繁搜索，减少了运行时开销。

其次，内存位置分配问题被抽象为二维条带打包问题，针对这一问题，本章给出了 MIP 和启发式算法两种求解方法。使得内存池数据摆放更加紧凑，内存碎片有效减少。

最后，不同于简单的内存复用技术，一个变量仅与另一个变量（一对一）共享内存。通过全局内存分配优化，这些块之间可以共享内存，而不会导致生命周期重叠，并且这种共享不限于一对一共享，同时也允许更高级的共享，以便一个大块可以与几个小块共享内存，反之亦然。本章的方法不需要人工干预，自动的在全局内存行为日志上生成优化后的内存分配计划，且对用户完全透明。

第6章 实验评估

本文在开源深度学习框架 TensorFlow 中实现了 METS，为了测试 METS 在深度学习训练中的内存占用优化情况和运行时开销。选取在深度学习领域有代表性的模型，包括了图像模型和文本模型。图像模型为图像分类网络，包括线性网络如 AlexNet 和 Vgg，也包括了非线性网络如 ResNet、InceptionV4 等；文本模型选取了，机器翻译中的 Seq2Seq 模型以及文本分类中的 BERT 模型进行测评。为了验证 METS 的有效性，我们使用不同的输入数据集，BatchSize 和模型层结构进行了基准测试。结果达到了本文的预期目标。

首先，我们将使用内存交换模块与不使用内存交换模块进行实验比较。其次，我们将使用内存分配优化模块与传统动态内存池技术进行比较。第三，我们将组合方法的性能与其他内存减少基准进行比较。最后我们还将与学术界已开源的内存优化技术进行比较。

6.1 实验平台

我们的评估是在具有 24GB GDDR5X 设备内存和 4608 个 CUDA 内核的 Nvidia Titan RTX GPU 上进行的。主机是 Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz，配备了 742GB DDR4 RAM。操作系统是带有 cuDNN-v7.6.5 和 CUDA-v10.2 的 Ubuntu 16.04.10 LTS。我们在第 4.2 节中通过离线分析来测量异构平台的拷贝传输速率和启动开销。在我们的实验平台上测得的 V_{hd} 是 0.1897， C_{in} 是 9335， V_{d2h} 是 0.1883， C_{out} 是 6374。TensorFlow 是一个流行的深度学习编程框架，专注于深度学习训练。TensorFlow 在内部优化了面向深度学习负载的训练速度和内存占用量，且被用户广泛使用，因此本文将其作为 Baseline。

6.2 实验设计原则

6.2.1 Benchmark 选择

为了证明 METS 可以在常见的 DNN 模型下显示出优秀的内存高效性，因此

这里筛选出一些典型的 DNN，包括 AlexNet, VGG, InceptionV4, ResNet 以及 Seq2Seq 和 BERT。VGG 是线性卷积神经网络，在向前和向后传播期间，其数据通常按顺序传播。与线性 DNN 相比，InceptionV4, ResNet 是非线性的。它们的层结构中有许多分支和连接。例如，就层结构而言，ResNet50 由 50 个残差块组成。每个残留层块包含两个卷积层，这些层块以前馈方式连接。另外，在每个残差块的输入和输出之间存在跳过连接。

Seq2Seq 本质上是一种 encoder-decoder 框架，以翻译任务中的“英译汉”为例，模型首先使用编码器对英文进行编码，得到英文的向量化表示 S ，然后使用解码器对 S 进行解码，得到对应的中文。由于 encoder 与 decoder 两端处理的都是序列数据，所以被称为 sequence-to-sequence，简称 Seq2Seq。另外，目前应用最多的编/解码器是 RNN (LSTM, GRU^[54])，但编/解码器并不限于 RNN，也有人拿 MLP 作为编码器。这里实验中配置的编/解码器为 LSTM。

BERT 模型是 Google 在 2018 年 10 月发布的语言表示模型，BERT 在 NLP 领域横扫了 11 项任务的最优结果，可以说是现今最近 NLP 中最重要的突破。BERT 模型的全称是 Bidirectional Encoder Representations from Transformers，是通过训练 Masked Language Model 和预测下一句任务得到的模型。这一模型具有很强的特征表达能力，已被用作各个自然语言处理任务的特征提取主干。

为了验证系统的通用性，对于数据集，我们选择一些轻量级的，例如 CIFAR-100（输入大小为 32x32）以及一些重量级的，例如 ImageNet（224x224）。对于文本模型 Seq2Seq 和 BERT，我们通过更改输入的隐藏层大小和序列长度来调整数据的规模。

表 6.1 实验选取 Benchmark

Table 6.1 Benchmark selected in the experiment

应用类型	计算机视觉		自然语言处理	
DNN 模型	线性网络	非线性网络	文本分类	机器翻译
	AlexNet VGG16	Inception V4 ResNet50	BERT	Seq2Seq
数据集	CIFAR、ImageNet		CNEWS	WMT16 English-German

6.2.2 实验设计

实验章节主要是为了回答三个问题，①METS 的整体内存使用效率和运行时开销情况是否优于目前的工业级编程框架？②自动内存交换模块是否可以有效降低内存负载峰值并减少内存占用？③全局内存分配优化模块是否可以有效减少内存碎片的产生并进一步减少内存占用？

因此，6.3 节选取了目前广泛应用的深度学习编程框架（TensorFlow, Pytorch, Mxnet）以及学术圈已经开源的优秀成果 SuperNeurons 进行对比，采用的 Benchmark 主要是为了覆盖线性卷积神经网络、非线性卷积神经网络。由于 METS 在内存占用和运行时开销上进行了优化，因此对比实验从训练速度和内存占用两个维度展开。

为了验证自动内存交换模块的优化效果，6.4 节设计了四组实验进行测评。由于自动内存交换模块的优化目标是有效降低内存负载的峰值，因此有必要在内存负载峰值的维度进行测评，第一组实验即在 Benchmark 的 6 个模型范围内对优化前和优化后的内存负载峰值进行比较实验。内存负载峰值降低的最终目标是为了有效减少内存占用，因此第二组实验完成了对 Benchmark 中各个模型内存占用的测评。前两组实验都是宏观的测评自动内存交换模块的效果，第三组实验从微观的动态内存负载曲线来观察内存负载的峰值是否被有效地降低。第四组实验从微观的动态内存占用曲线来观察最终内存的占用值是否有效减少。

为了验证全局内存分配优化模块的效果，6.5 设计了 5 组实验进行测评。首先，全局内存分配优化一定会带来内存占用的减少，为了验证这一效果，第一组实验针对 Benchmark 中的模型，展开优化前和优化后的内存占用对比，来验证内存优化的效果。其次，由于减少了在内存池中检索可用内存块的开销，METS 的运行时开销得到了减少，本节设计第二组实验验证使用内存分配优化模块前后，运行时开销的变化情况。第三，由于内存分配优化模块是基于启发式算法进行求解的，启发式算法的开销情况非常值得关注，因此第三组实验用于测评启发式算法的求解开销是否是可接受的。第四，卷积工作区间动态分配策略可以有效规避卷积工作区的内存占用，这一策略是否可以进一步降低内存占用值的问题交给第四组实验来解决。最后，由于内存分配优化模块是为了减少池内的碎片，第五组实验从

甘特图的角度观察内存碎片的数量是否在优化之后有效地减少。

6.3 METS 总体评测

在本节中，我们评估 METS 的总体内存占用和训练速度情况。证明自动内存交换和内存分配优化组合方法相比于原始版本的优化效果。我们使用图像数据集 ImageNet，并使用不同类别的常用 Benchmark (ResNet50、VGG16、InceptionV4 和 Seq2Seq) 评估。AlexNet 与 VGG16 同为线性卷积神经网络，此处略去。由于篇幅所限，我们仅通过扫描批处理大小来报告它们的峰值内存占用量和相应的训练速度。

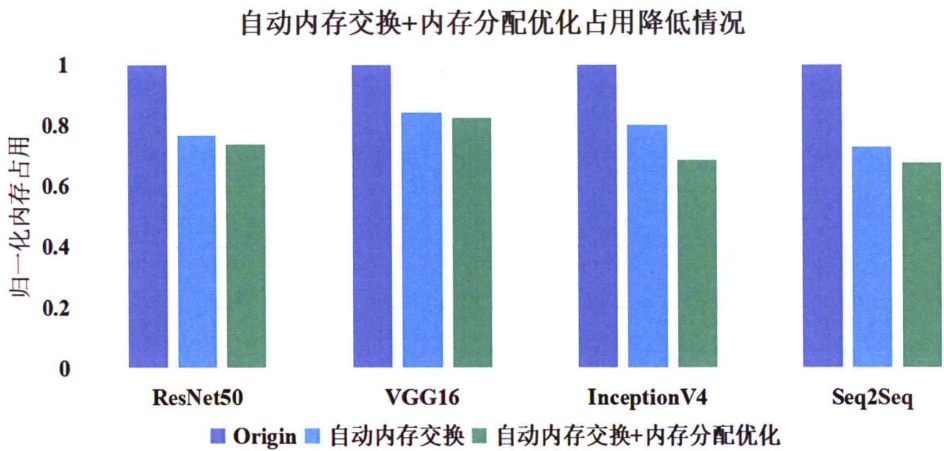


图 6.1 METS 内存占用优化结果

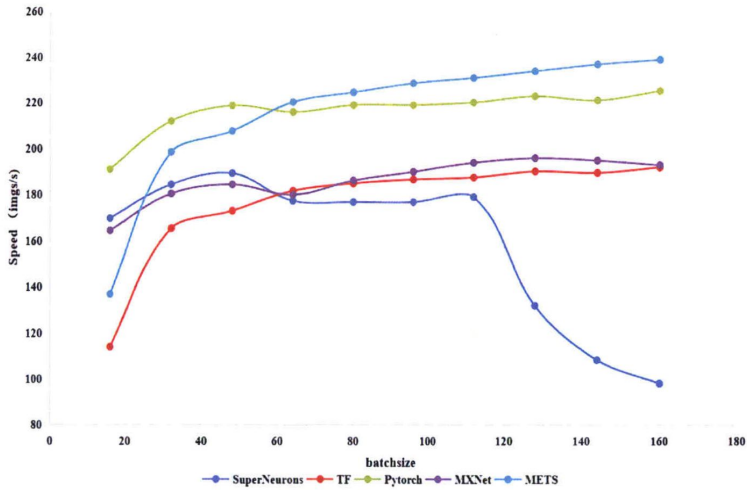
Figure6.1 Optimization result of METS memory usage

如图 6.1 所示，横轴为四种不同的模型，纵轴为归一化的内存占用值；蓝色柱状体为原始版本的内存占用，红色柱状体为自动内存交换模块的内存占用，绿色柱状体为自动内存交换和内存分配优化相结合的内存占用。当前数据是在 BatchSize 为 64 的配置下得到的。可以观察到，各个模型的自动内存交换和内存分配优化策略的结合可以进一步地降低内存占用，缓解 GPU 的内存使用压力。

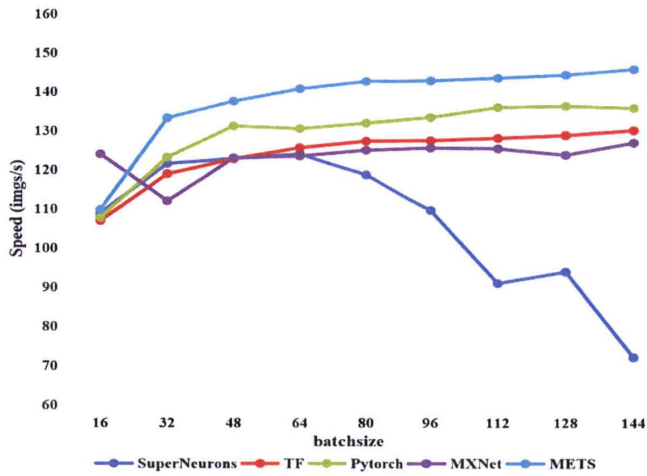
6.3.1 训练速度对比

训练的速度是深度学习训练系统的核心性能指标。由于自动内存交换模块不增加运行时开销，且内存分配优化算法的运行时开销相比于原始版本有所降低，因此在整体上本文提出的 METS 在训练速度上具有优势。这里将 METS 的训练速

度与常见的深度学习框架如 TF, Pytorch, MXNet 作比较, 除此之外, 本节也跟学术界已开源的内存优化解决方案 SuperNeurons 进行了比较。由于 SuperNeurons 的开源实现中不包含 RNN 和 Transformer 的实现, 为了保证公平对比, 本组实验只选用了 CNN 模型。



(a)



(b)

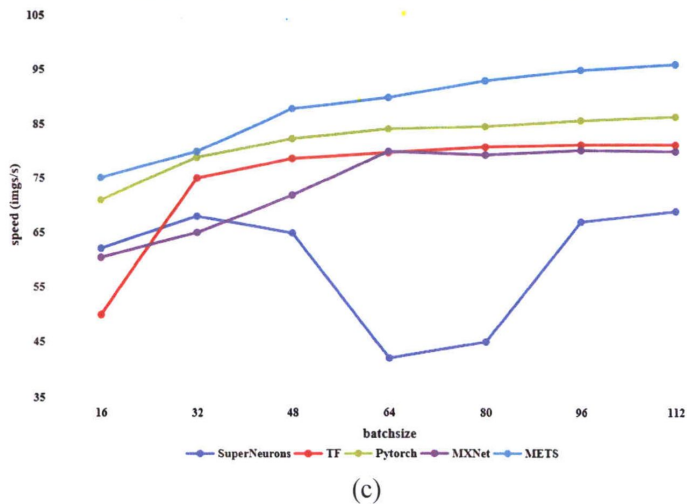


图 6.2 开源解决方案训练速度对比

Figure6.2 Open source solution training speed comparison

如图 6.3 所示为 METS 与 TensorFlow、Pytorch、MXNet 和开源解决方案 SuperNeurons 的训练速度对比。横轴为不同的 BatchSize 配置，纵轴为一次迭代训练中的训练速度单位是 imgs/s。图 6.3(a)为 ResNet50 的实验结果，图 6.3(b)为 VGG16 的实验结果，图 6.3(c)为 InceptionV4 的实验结果。METS 的训练速度在这三个示例的三种 BatchSize 配置下相比于其他解决方案具有优势。一个有趣的发现是，与其他方案相比，SuperNeurons 令人失望。这并不奇怪，因为 SuperNeurons 是通过手写 C ++和 cuDNN 实现的，无法与基于工业解决方案的其他方案相提并论。

6.3.2 内存占用对比

如图 6.2 所示为 METS 与 TensorFlow、Pytorch、MXNet 和开源解决方案 SuperNeurons 的内存优化情况。图 6.2(a)使用的 benchmark 为 VGG16，图 6.2(b)使用的 benchmark 为 ResNet50。横轴为不同的 BatchSize 配置，纵轴为一次迭代训练中的内存占用。可以看到在减少内存占用方面，METS 优于其他解决方案。SuperNeurons 内存占用仅仅比 TensorFlow 低，主要是由于 SuperNeurons 是 2018 年发表的论文，开源深度学习框架后期在内存管理上做了更多的优化，导致 SuperNeurons 的内存优化优势不明显。

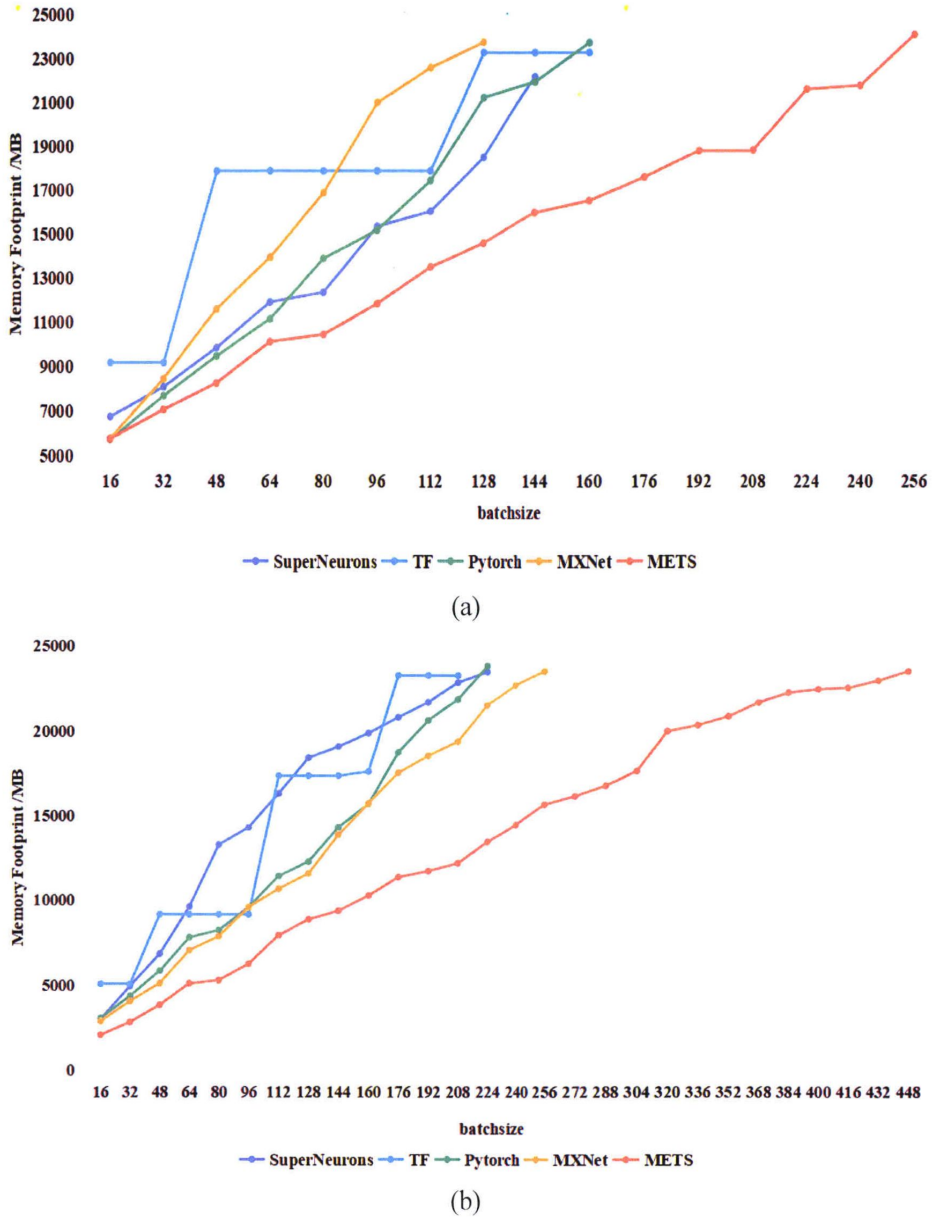


图 6.3 开源解决方案内存占用对比

Figure6.3 Comparison of memory footprint of open source solutions

6.4 基于代价模型的自动内存交换实验评测

在本小节中，我们评估了自动内存交换模块内存负载峰值和内存占用的降低情况。通过对比实验，验证自动内存交换模块的效果，包括内存负载降低情况以及内存占用降低情况。请注意，内存负载峰值降低与内存占用降低情况有所差异，

内存负载峰值指的是在模型训练过程中，驻留在 GPU 内的数据量总和的峰值。而内存占用为深度学习训练系统占用的 GPU 内存总量，理想情况下，内存负载的峰值即为内存占用的值，但是在实际运行过程中，往往会存在内存碎片和占用的内存不能被百分之百利用的情况。因此内存占用总是大于内存负载峰值，由于自动内存交换技术主要是为了通过降低内存负载峰值来减少内存占用。因此有必要对内存负载峰值和内存占用的降低情况都进行评估。

6.4.1 自动内存交换模块的内存负载和内存占用收益

如图 6.4 所示，为自动内存交换模块的内存负载峰值降低效果示意图。横轴为不同的测试模型，纵轴为归一化的内存负载峰值。Origin 为未使用内存交换模块的对照组，其内存管理模块为默认的动态内存池，Opt 为使用了自动内存交换模块的实验组，为了验证自动内存交换模块的效果，这里实验组也使用动态内存池做内存管理。由于用户的输入的内存负载上限这一超参数会影响到实际运行中负载的峰值，实验组通过多次调整超参数，来找到不增加额外运行时开销的最大负载上限值。

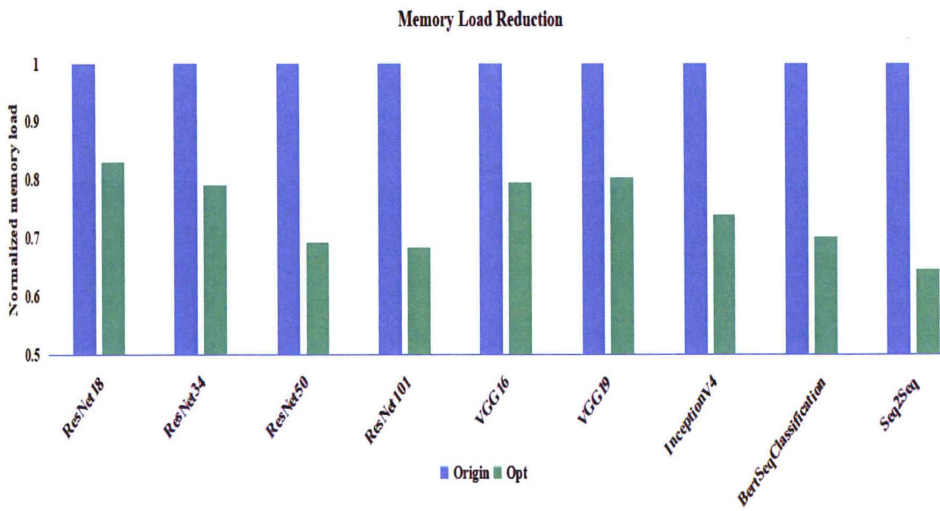


图 6.4 自动内存交换模块内存负载收益

Figure6.4 Automatic memory swap module memory load gain

ResNet 和 VGG 使用 CIFAR100 数据集(图像尺寸为 32*32)，训练的批处理大小为 500，InceptionV4 的图像尺寸为 299*299，训练的批处理大小为 32。文本语言模型方面，选取了文本分类任务中的 BERT 模型以及 Seq2Seq 机器翻译模型进

行实验，BERT 模型的最大文本长度为 32，训练的批处理大小也为 32，Seq2Seq 模型的 Embedding 维度为 200，训练批处理大小为 16。从图 6.5 中可以观察到在各个模型的配置下，自动内存调度算法都降低了内存的负载峰值，并且在内存占用上也获得了收益。但是内存负载峰值的降低幅度总是大于内存占用的降低，表明使用传统的内存池技术，存在一定的内存碎片问题。另一个观察是，网络结构越深，代表模型训练单次迭代周期越长，自动内存交换模块对这样的模型优化效果越好。这表明内存块的重用距离被有效地利用。换出的内存块可以被其他的内存块利用，使得内存的使用效率有效提升。

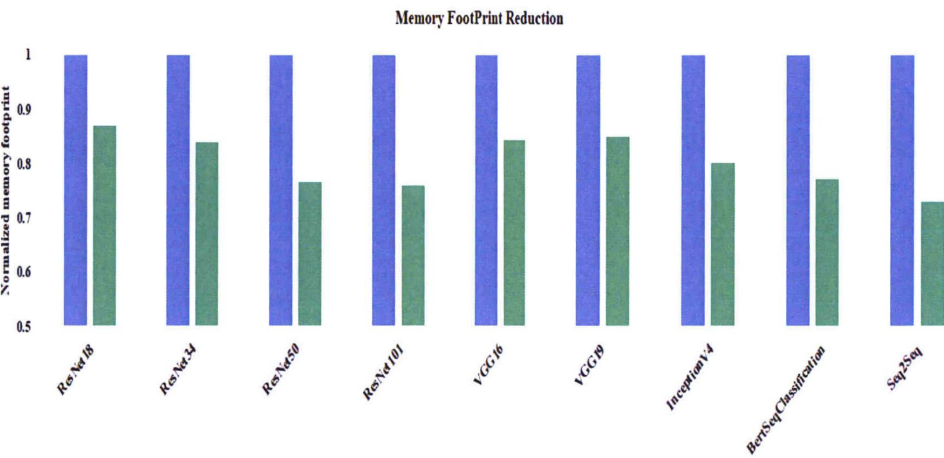


图 6.5 自动内存交换模块内存占用收益

Figure6.5 Memory footprint reduction of automatic memory swap module

6.4.2 一次迭代内存负载变化分析

为了验证自动内存交换模块的有效性，这里给出了 ResNet18 训练过程中的内存负载变化情况，实验配置中图像输入尺寸为 224*224，训练的 BatchSize 为 64，横轴表示训练过程中的内存行为索引，纵轴为内存负载的值。在此图中，用户设置的内存负载上限是 5000MB，可以看到，在使用自动内存交换模块之后，内存负载峰值始终被压缩在 5000MB 以下。图中表示的是模型训练五次迭代的内存负载变化示意图，图中有 7 个负载峰值，前两个负载峰值是训练系统初始化的消耗，后 5 个为五次训练迭代的内存负载变化曲线。

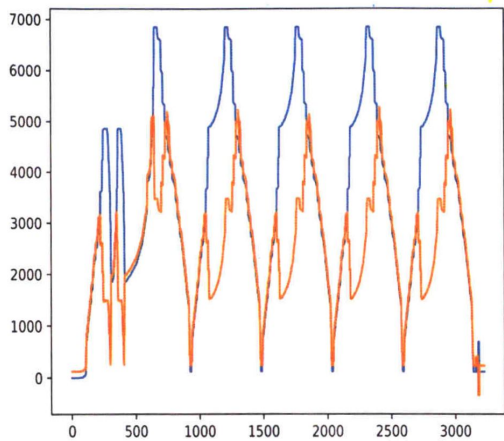


图 6.6 ResNet18 训练一次迭代的内存负载变化曲线

Figure6.6 Memory load curve of five iteration of ResNet18 training

通常负载峰值出现在模型反向传播的开始阶段，蓝色的折现为未使用自动内存交换模块的优化前版本，黄色则为使用了自动内存交换模块的版本。从蓝色图线中可以观察到，由于前向计算不断生成新的特征图，因此其内存负载不断上升，在反向计算开始后，随着反向传播的进行，特征图和梯度图不断被释放，可以观察到内存负载的快速下降。从优化后的黄色图线中可以看到，随着前向计算的进行，内存块不断地被交换到 CPU 主存，有效地降低了内存负载，并在反向传播的过程中，被及时地交换回 GPU，因此在图线中出现了两个负载峰值。

6.4.3 一次迭代内存占用变化分析

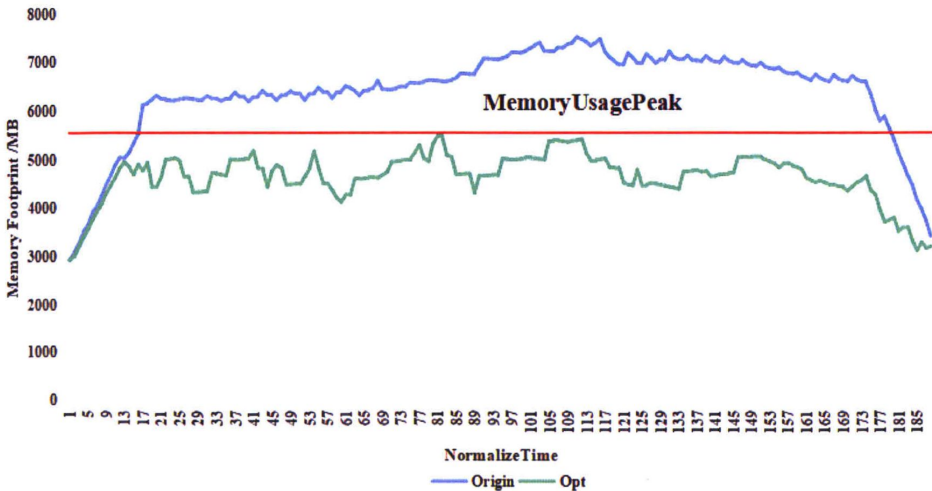


图 6.7 ResNet18 一次训练迭代的内存占用曲线

Figure6.7 Memory usage curve of one training iteration of ResNet18

为了验证自动内存交换模块对内存占用的降低效果，这里同样给出了更加细粒度的刻画。如图所示是 ResNet18 在训练中一次迭代的内存占用变化。横轴为归一化的时间坐标，纵轴为 GPU 内存占用大小。蓝色的线表示优化前内存占用曲线，红色的线表示自动内存交换模块的内存占用图线。首先可以观察到优化后的内存占用相比于优化前有明显地降低，并且内存负载峰值的下降也直接带来了内存占用的降低。红色的横线表示的是优化后内存使用的峰值，这一峰值大于用户设置的内存负载上限 5000MB，表明内存占用的峰值大于内存负载的峰值，尽管自动内存交换模块有效地降低了内存负载峰值，但是还是存在内存碎片问题，使得内存负载的值小于实际内存占用的值。

6.5 基于全局访存行为的内存分配优化实验评测

本节为了验证内存分配优化策略的有效性，比较了使用内存分配策略和不使用内存分配优化策略的 GPU 内存占用和一次迭代运行时间，在后续图表中用 origin 表示，以及我们的优化版本用 opt 表示，适用于三个 CNN (VGG16, ResNet50, InceptionV4) 和一个 RNN (seq2seq)。CNN 的训练使用 32、64 和 128 的批处理大小，而 seq2seq 的训练使用 32、64、128 和 256 的批处理大小。我们使用 ImageNet 和来自 WMT14 的英德语料库分别作为 CNN 和 seq2seq 的数据集。我们使用前 10 个 mini-batch 进行预热，然后使用 20 个 mini-batch 进行评估。为了独立测评内存分配优化策略的有效性，在本节的实验中，关闭了自动内存交换模块，打开了 NVIDIA CUDA 的统一内存，这使我们可以在 Profiler 的运行中获得比物理容量更多的内存，但在运行时间测量的实验中将其关闭，因为统一内存会导致性能开销。

6.5.1 内存分配优化算法内存降低

首先，在本节中证明了，与基于池的方法相比，我们的方法减少了更多的内存。其次是关于卷积算法的。有许多用于计算卷积的算法，内存效率最高的算法仅需要用于输入和输出的内存，但是我们可以通过分配额外的临时内存（称为工作空间）来更快地计算卷积。本节将验证卷积工作空间动态分配的实际收益。本节还评估了在 5.4 中实现启发式算法。不仅将一小时内将启发式解决方案与 CPLEX 12.8 找到的最佳解决方案进行比较，还比较了不同配置的启发式算法的计

算时间。

如图 6.5 所示为 InceptionV4、VGG16、Bert 和 Seq2Seq 的内存占用降低情况。AlexNet 与 VGG16 同为线性卷积神经网络，此处略去。横轴为各个模型的 BatchSize 大小，纵轴为模型单次迭代内存占用量。条形图中蓝色的柱状体代表未使用内存分配优化策略的对照组，红色柱状体代表使用内存分配优化策略的实验组。可以观察到 CNN 和 RNN 的内存占用都有不同程度地降低，表明我们的优化在所有模型中均有效。其中含有复杂非线性连接的 InceptionV4 的优化效果明显高于 VGG16，ResNet50 等非线性连接较少的模型。这表明，动态内存池技术在面对复杂多变的上层应用结构时，难以适应变化的内存请求，容易出现内存碎片，这一问题可以通过 Profile 引导的全局内存分配优化技术得到解决。图中 InceptionV4 在 128 的批处理大小下，Baseline 所需的内存空间超过了 GPU 的物理内存容量，蓝色的内存占用值是通过 BatchSize 预估的，通过全局内存分配优化之后，可以被运行在 24GB 内存的 GPU 上。这一范例展现了内存分配优化的有效性。

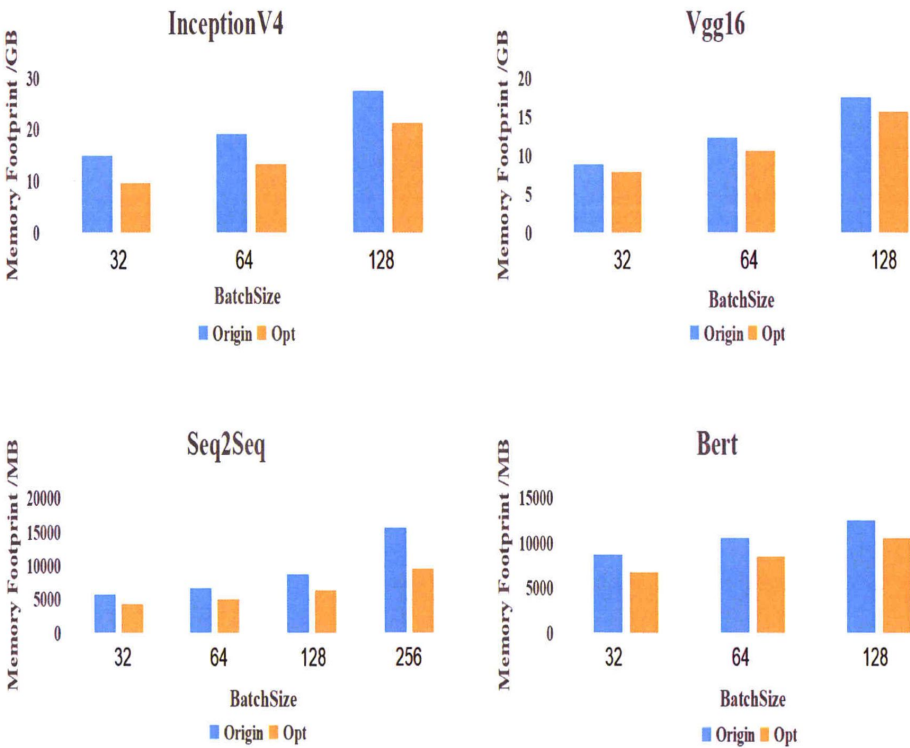


图 6.8 全局内存分配优化内存占用降低

Figure6.8 Global memory allocation optimization memory footprint reduction

6.5.2 内存分配优化算法训练耗时评估

CNN 模型每个 batch 的平均训练时间记录在图 6.6 中，其中“N / A”表示由于内存不足，无法训练模型。AlexNet 与 VGG16 同为线性卷积神经网络，此处略去。结果表明，即使使用相同的 BatchSize，我们的方法也可以加速某些模型（Vgg16，ResNet50 和 InceptionV4）的训练。这是因为优化的版本分配内存的速度非常快。给定内存请求，Baseline 从内存池中搜索可用的，足够大小的内存块，并且该内存搜索的运行成本随着池中内存块数量的增加而增加。相反，优化版本仅返回训练前计算好的内存地址。这在 InceptionV4 中尤为明显，在 InceptionV4 中，BatchSize 为 32 的配置下，优化版本的训练速度是原始版本的 1.32 倍。图 6.5 中还显示，我们的方法允许使用更大的 BatchSize，这使我们能够更充分地利用 GPU。例如，优化版本能够在 BatchSize 为 128 的配置下正常训练，而原始版本无法在同样内存容量的 GPU 上训练。

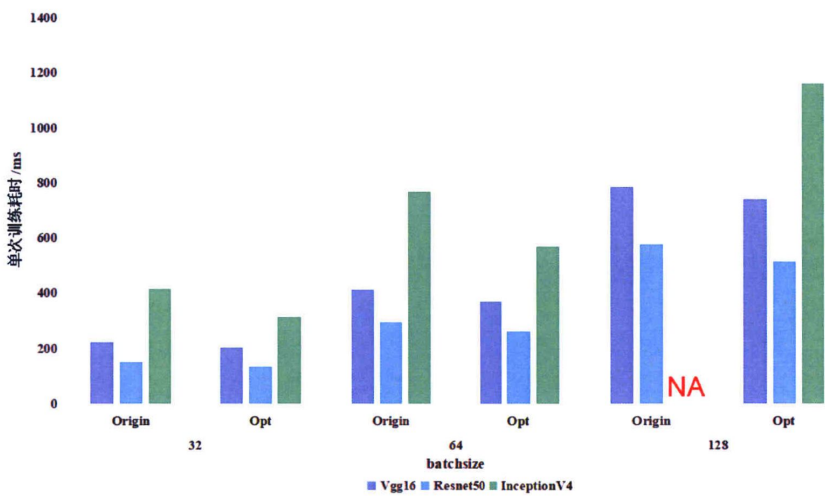


图 6.9 CNN 模型运行时开销降低情况

Figure6.9 CNN model runtime overhead reduction

RNN 模型 Seq2Seq 和近来非常流行的 BERT 的平均训练时间如图所示。BatchSize 的配置覆盖 32、64、128 和 256 四种。在四个训练 BatchSizeSize 配置下，优化版本下 Seq2Seq 和 BERT 的平均训练时间都小于原始版本。128 和 256 的 BatchSize 下，优化版本中的训练变得比原始版本中的训练更快。我们猜测这

是由于基于池的内存管理系统浪费了 GPU 内存。当由于可用内存不足而无法分配请求的内存时，基于池的内存管理系统将释放所有未使用的内存块，并从物理 GPU 内存分配后续的请求内存，这比复用池中的内存块具有更高的运行时成本。

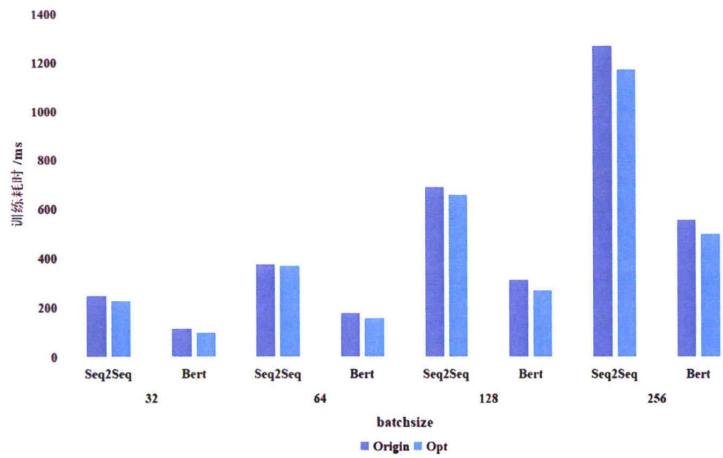


图 6.10 Seq2Seq 和 BERT 的运行时空降低情况

Figure6.10 Seq2Seq and BERT training time reduction

6.5.3 启发式算法开销评估

经过实际测试，CPLEX 只能在 MLP 训练和 AlexNet 以及 Vgg16 的推理任务上获得结果。其他的实验配置下，由于问题规模过大，CPLEX 无法在可接受的时间内得到最优解。为了验证本文提出的启发式算法可以得到足够相近的近似解。此处给出了整数线性规划方法(MILP)和 Best-Fit 启发式算法在上述三个任务中的结果。

表 6.2 MILP 与 Best-Fit 启发式算法对比

Table 6.2 Comparison of MILP and Best-Fit heuristic algorithms

测试模型	MLP 训练		AlexNet 推理		VGG16 推理	
	MILP	启发式算法	MILP	启发式算法	MILP	启发式算法
内存占用	1198	1169	1992	1966	10150	9994
	MB	MB	MB	MB	MB	MB
归一化比例	1	0.976	1	0.987	1	0.984

如表 6.2 所示，启发式算法和基于 MILP 的求解方法在，上述三个任务中都得到了非常接近的优化效果，表明 Best-Fit 启发式算法能够得到良好的近似解，同时由于 CPLEX 求解非常耗时，因此系统默认采用启发式算法做全局的内存分配

优化求解。

如表 6.3 所示，启发式算法求解 Benchmark 中模型训练问题的执行时间都在 5s 以内，这表明启发式算法可以足够快地进行实际应用。这里应该注意，启发式算法的计算时间与模型训练所需的内存块数量相关，而与模型训练的 BatchSize 无关，BatchSize 只能改变训练过程中单个内存块的大小，而不能改变训练过程中的内存块数量，内存块的数量只与模型结构相关。

表 6.3 Best-Fit 启发式算法执行耗时

Table6.2 Best-Fit heuristic algorithm execution time-consuming

训练任务	Best-Fit 启发式算法耗时
Vgg16(BatchSize_64)	1.10s
ResNet50(BatchSize_64)	1.39s
InceptionV4(BatchSize_64)	4.32s
Seq2Seq(BatchSize_16/embed_dim_200)	0.98s
BertForSeqClassification(max_length_32/ BatchSize_32)	2.58s

6.5.4 动态卷积工作空间分配测评

为了验证动态卷积工作空间分配的效果。本小节以 ResNet50 和 InceptionV4 为例，给出了这两种模型在三个 BatchSize 下的内存占用情况。

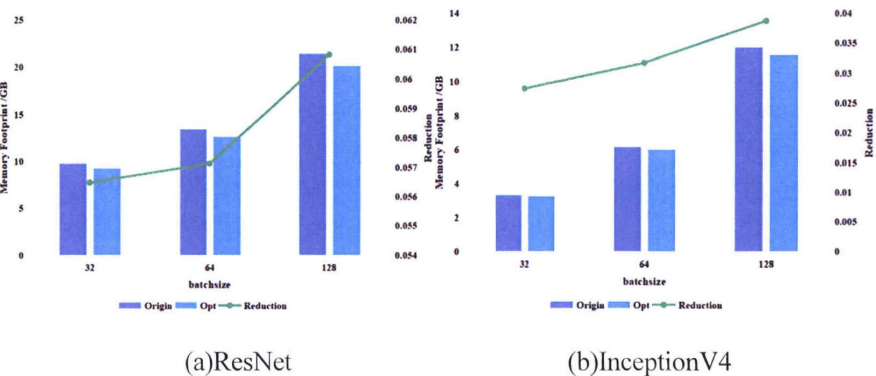


图 6.11 动态卷积工作空间分配优化效果

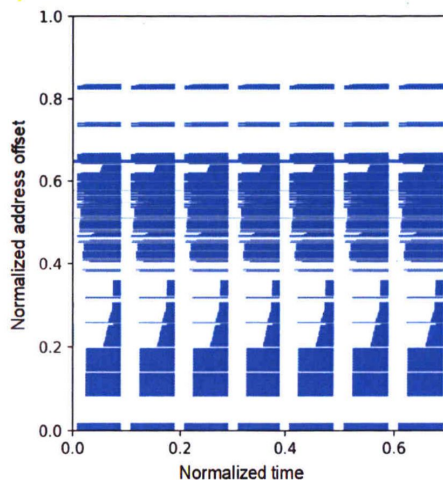
Figure6.11 The memory footprint reduction of dynamic convolution work space allocation

如图所示，图 6.11(a)为 ResNet 的实验结果，图 6.8(b)为 InceptionV4 的实验结

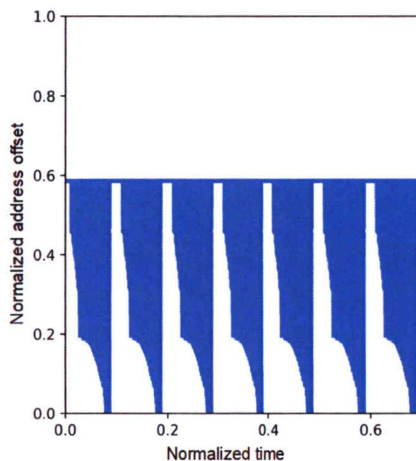
果，蓝色柱状体是未使用动态卷积工作空间分配算法的原始版本，而红色柱状体为使用动态卷积工作空间的优化版本。绿色的图线表示的是，优化版本相对于原始版本的内存占用减少。从实验结果中可以观察到，使用静态分配与动态分配相结合的方式处理内存请求可以进一步降低内存占用。训练系统运行的整个过程中，静态分配内存从低地址到高地址进行分配，而卷积的工作空间由高地址到低地址分配。由于在训练过程中，同一时间内只能有一个卷积算子运行，所以在内存池的高地址段将始终只有一个内存块存在，实现了卷积工作空间之间内存复用。同时，内存分配优化算法可以不再计入卷积工作空间，更加便于其压缩内存碎片。

6.5.5 内存碎片降低情况

为了验证本文的方法是否能够有效地降低内存碎片，我们在 ResNet18 工作负载训练中绘制了前七轮迭代的甘特图。甘特图的横轴是归一化的时间轴，纵轴是归一化的地址偏移，指的是内存块的地址与内存池起始地址的偏移量。图中的每个矩形代表对 GPU 的设备内存中某个内存块的内存访问行为。矩形的宽度是从分配时间到当前存储块的空闲时间所经过的时间，可以视为当前内存块的生命期。矩形的高度表示当前存储块的大小。蓝色矩形之间的空白区域可以被看作是内存碎片，图(a)是原始版本的甘特图，图(b)是优化版本的甘特图，可以看到原始版本相比于优化版本存在更多的内存碎片，通过内存分配优化算法，系统可以根据内存块的生存期和大小来更智能地分配内存，实现出色的内存共享。几个较小的内存块可以共享较大变量的内存，反之亦然。由于优化版本提供了有效的内存共享，内存块的摆放更加紧凑，内存碎片的空间也被有效的压缩。



(a)Origin



(b)Opt

图 6.12 全局内存分配优化前后甘特图对比

Figure6.12 Gantt chart comparison before and after global memory allocation optimization

6.6 本章小结

本章对 METS 的实验测评进行了详细地介绍和分析。首先，在 6.1 节我们介绍了实验所用到的软硬件环境，其次在 6.2 节中介绍了实验所涉及的 Benchmark。6.3 节从总体上将本系统与开源解决方案进行对比，从内存占用和训练速度的维度上验证本文系统的有效性。6.4 节主要分析自动内存交换模块的效果，包括其对内存负载的降低效果以及内存占用的降低效果。6.5 节分析了全局内存分配优化模块的运行时开销情况以及内存消耗情况。

实验结果表明，本系统在训练速度上相比与开源解决方案有明显优势，同时在内存占用上可以获得显著降低；并且提高了有限内存容量下，模型可以训练的最大 batchsize。相比于 TensorFlow，METS 在不考虑时间开销的情况下，允许以 1.6 倍最大 batchsize 训练 VGG16，以 2.1 倍最大 batchsize 训练 ResNet50。

本章还对自动内存交换模块和全局内存分配优化模块分别进行了实验评估，自动内存交换模块将内存负载和内存占用都降低了 20%-30%。全局内存分配优化模块可以将内存占用降低 10-20%，并且减少运行时的开销。两个模块的正交组合可以进一步压缩内存占用的大小。

第7章 总结与展望

7.1 本文工作总结

GPU（图形处理单元）已用于许多数据密集型应用程序。其中，深度学习是当今 GPU 最重要的消费类应用之一，而深度学习训练场景是 GPU 内存消耗最大的应用场景。随着深度学习应用算法采用更深，更大的模型以实现更高的准确性，而 GPU 内存容量增长趋缓，内存优化已成为深度学习系统的重要研究课题。

在本文中，我们首先对深度学习训练系统的内存行为进行详细的刻画，并指出了现有深度学习内存管理技术的内存利用率低和内存使用效率低的问题。在此基础上，本文提出了内存高效训练系统 METS，并详细介绍了系统的框架结构以及执行流程。在优化方法上，借助于深度学习训练中的迭代性质，本文提出了两种正交的内存优化方法：基于代价模块的自动内存交换和基于全局访存行为的内存分配优化，以有效地减少深度学习训练系统中 GPU 的内存消耗。

为了解决深度学习训练系统中内存使用低效的问题，本文提出了基于代价模型的自动内存交换技术。这一技术借助了 DNN 训练中的迭代特性，可以自动跟踪 DNN 工作负载的前期内存行为并在后续迭代中安排调度内存交换执行，以降低 GPU 的内存压力。由于借助了前几轮迭代的内存行为日志，系统无需感知上层的计算图语义即可生成内存交换计划。实验表明，自动内存交换技术通过将当前未使用的内存块换出到 CPU 内存中，从而以理想的低通信开销有效地减少了内存负载。

为了解决深度学习训练系统中内存利用率低的问题，本文提出了针对 DNN 的 Profile 引导的内存优化技术，基于全局访存行为进行内存分配优化。系统通过提取一次迭代过程中所有内存块的生命周期及其大小，可以将全局的内存分配优化问题抽象为二维条带打包问题的特殊情况。这一问题是 NP 难的，本文给出了求解这一问题最优解的混合整数线性规划解法，开发了一种简单的启发式算法，以获取高效，快速的内存分配近似解。通过实验证实了我们的方法减少了内存占用，并在图像 CNN 模型和文本模型加快了训练的计算速度。内存分配优化技术

有效地压缩了内存碎片，并且优化了内存池中内存块的摆放，提高了 GPU 内存的利用率。

这两种技术对最终用户是透明的，不需要人工干预且相互正交。实验证明，二者的组合可以进一步的降低内存占用，并且在内存负载上限设置宽松的情况下，有效降低了运行时开销。

本文提出的 METS 系统的主要优势包括了三方面：

首先，**内存占用的降低**，在不过度交换内存的情况下满足了物理内存容量要求，避免了冗余数据通信。

其次，**对用户透明**，并在早期迭代时自动开始工作。它不需要用户了解网络的内存消耗，也不需要决定需要交换的层有哪些。用户不需要精心设计内存分配的策略，系统可以自动根据 profile 完成内存块的合理分配。

最后，**在模型复杂度和 batchsize 的选择上具有更大的可伸缩性**。由于我们可以在更大的范围上改变 DNN 的模型类型和深度，batchsize 大小，因此在给定内存容量限制的前提下，在 batchsize 的选择和模型复杂度的权衡上给予用户更大的空间。如图 7.1 所示。

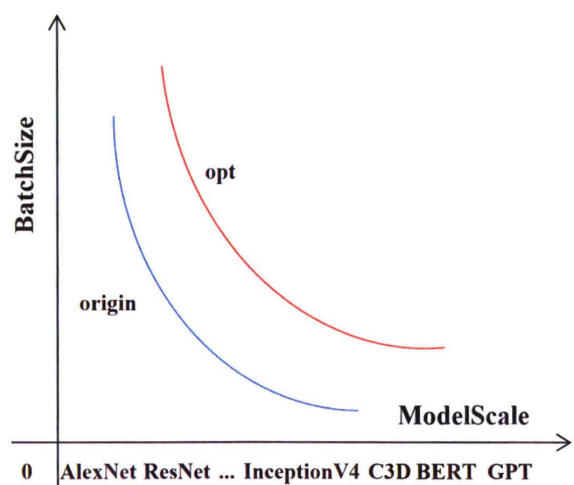


图 7.1 METS 优化效果总体示意

Figure7.1 METS optimization effect overview

7.2 下一步研究方向

虽然本文针对深度学习训练系统的内存使用进行了针对性的解决方案，但仍存在一些值得改进和优化的方向：

- **面向不具备迭代特征应用的扩展优化**，其内存占用降低将是一个值得研究的课题，或许可以针对迭代变化应用中迭代不变的内存行为进行优化，而将迭代变化的访存行为不做干预。我们不会在模型训练的迭代变化部分中优化请求的内存使用率。为此，可以提供两个操作，即中断和恢复，在内存行为进入迭代变化区域时 Profiler 中断，并在内存行为进入迭代不变时恢复 Profiler 的监控。以期部分缓解内存占用。
- **探索分布式环境中的内存减少**，深度学习训练在工业界往往不是单节点训练，而是采用多节点的数据并行训练。这一方法也是训练大规模深度学习模型的常用技术方案。数据并行的分布式训练可以降低单卡训练的内存压力，本文的方法则可以进一步的减小单节点训练的内存占用，并提高单节点可训练的批处理大小。同时，由于数据并行分布式训练需要进行参数的同步，这一同步过程容易产生可观的通信开销，因此可以考虑使用 RDMA 进行数据传输，来减小这一通信开销。本文提出的内存高效训练系统与 RDMA 的配合之下，可以尝试在有限的内存资源下，训练层数更多，特征维度更高的深度学习模型。

参考文献

- [1] Geoffrey E, Hinton, Simon, et al. A fast learning algorithm for deep belief nets.[J]. Neural computation, 2006.
- [2] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks[J]. Advances in neural information processing systems, 2012, 25: 1097-1105.
- [3] Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, inception-resnet and the impact of residual connections on learning[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2017, 31(1).
- [4] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate[J]. arXiv preprint arXiv:1409.0473, 2014.
- [5] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[J]. arXiv preprint arXiv:1409.1556, 2014.
- [6] Choquette J, Giroux O, Foley D. Volta: Performance and programmability[J]. Ieee Micro, 2018, 38(2): 42-52.
- [7] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [8] Bottou L. Stochastic gradient descent tricks[M]//Neural networks: Tricks of the trade. Springer, Berlin, Heidelberg, 2012: 421-436.
- [9] Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2015: 1-9.
- [10] Sutskever I, Martens J, Dahl G, et al. On the importance of initialization and momentum in deep learning[C]//International conference on machine learning. PMLR, 2013: 1139-1147.
- [11] Duchi J, Hazan E, Singer Y. Adaptive subgradient methods for online learning and stochastic optimization[J]. Journal of machine learning research, 2011, 12(7).
- [12] Lavin A, Gray S. Fast algorithms for convolutional neural networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 4013-4021.
- [13] Chetlur S, Woolley C, Vandermersch P, et al. cudnn: Efficient primitives for deep learning[J]. arXiv preprint arXiv:1410.0759, 2014.

-
- [14] NVIDIA, “cnmem,”<https://github.com/NVIDIA/cnmem>, 2015.
- [15] Bastien F, Lamblin P, Pascanu R, et al. Theano: new features and speed improvements[J]. arXiv preprint arXiv:1211.5590, 2012.
- [16] Abadi M, Barham P, Chen J, et al. Tensorflow: A system for large-scale machine learning[C]//12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16). 2016: 265-283.
- [17] Tokui S, Oono K, Hido S, et al. Chainer: a next-generation open source framework for deep learning[C]//Proceedings of workshop on machine learning systems (LearningSys) in the twenty-ninth annual conference on neural information processing systems (NIPS). 2015, 5: 1-6.
- [18] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional architecture for fast feature embedding[C]//Proceedings of the 22nd ACM international conference on Multimedia. 2014: 675-678.
- [19] Paszke A, Gross S, Massa F, et al. Pytorch: An imperative style, high-performance deep learning library[J]. arXiv preprint arXiv:1912.01703, 2019.
- [20] Chen T, Li M, Li Y, et al. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems[J]. arXiv preprint arXiv:1512.01274, 2015.
- [21] Zaharia M, Chowdhury M, Franklin M J, et al. Spark: Cluster computing with working sets[J]. HotCloud, 2010, 10(10-10): 95.
- [22] Chen T, Xu B, Zhang C, et al. Training deep nets with sublinear memory cost[J]. arXiv preprint arXiv:1604.06174, 2016.
- [23] Meng C, Sun M, Yang J, et al. Training deeper models by GPU memory optimization on TensorFlow[C]//Proc. of ML Systems Workshop in NIPS. 2017, 7.
- [24] Shirahata K, Tomita Y, Ike A. Memory reduction method for deep neural network training[C]//2016 IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP). IEEE, 2016: 1-6.
- [25] Han S, Pool J, Tran J, et al. Learning both weights and connections for efficient neural networks[J]. arXiv preprint arXiv:1506.02626, 2015.
- [26] Gong Y, Liu L, Yang M, et al. Compressing deep convolutional networks using vector quantization[J]. arXiv preprint arXiv:1412.6115, 2014.
- [27] Jia Z, Padon O, Thomas J, et al. TASO: optimizing deep learning computation with automatic generation of graph substitutions[C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 47-62.
- [28] He B, Yu J X. High-throughput transaction executions on graphics processors[J].

arXiv preprint arXiv:1103.3105, 2011.

- [29] Cui H, Zhang H, Ganger G R, et al. Geeps: Scalable deep learning on distributed gpus with a gpu-specialized parameter server[C]//Proceedings of the Eleventh European Conference on Computer Systems. 2016: 1-16.
- [30] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design[C]//2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). IEEE, 2016: 1-13.
- [31] Shriram S B , Garg A , Kulkarni P . Dynamic Memory Management for GPU-Based Training of Deep Neural Networks[C]// 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 2019.
- [32] Wang L, Ye J, Zhao Y, et al. Superneurons: Dynamic GPU memory management for training deep neural networks[C]//Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming. 2018: 41-53.
- [33] Le T D , Imai H , Negishi Y , et al. TFLMS: Large Model Support in TensorFlow by Graph Rewriting[J]. 2018.
- [34] Huang C C, Jin G, Li J. SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. 2020: 1341-1355.
- [35] Gruslys A, Munos R, Danihelka I, et al. Memory-efficient backpropagation through time[J]. arXiv preprint arXiv:1606.03401, 2016.
- [36] Peng X, Shi X, Dai H, et al. Capuchin: Tensor-based GPU memory management for deep learning[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. 2020: 891-905.
- [37] Wahib M, Zhang H, Nguyen T T, et al. Scaling distributed deep learning workloads beyond the memory capacity with KARMA[J]. arXiv preprint arXiv:2008.11421, 2020.
- [38] Pleiss G, Chen D, Huang G, et al. Memory-efficient implementation of dense-nets[J]. arXiv preprint arXiv:1707.06990, 2017.
- [39] Foley D, Danskin J. Ultra-performance Pascal GPU and NVLink interconnect[J]. IEEE Micro, 2017, 37(2): 7-17.
- [40] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition

tion. 2016: 770-778.

[41] Goodfellow I J, Pouget-Abadie J, Mirza M, et al. Generative adversarial networks[J]. arXiv preprint arXiv:1406.2661, 2014.

[42] Zaremba W, Sutskever I, Vinyals O. Recurrent neural network regularization[J]. arXiv preprint arXiv:1409.2329, 2014.

[43] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735-1780.

[44] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018.

[45] Sutskever I, Vinyals O, Le Q V. Sequence to sequence learning with neural networks[J]. arXiv preprint arXiv:1409.3215, 2014.

[46] Hasan Y, Chang M. A study of best-fit memory allocators[J]. Computer Languages, Systems & Structures, 2005, 31(1): 35-48.

[47] Lewis H R. Computers and intractability. A guide to the theory of NP-completeness[J]. 1983.

[48] Arahori Y, Imamichi T, Nagamochi H. An exact strip packing algorithm based on canonical forms[J]. Computers & Operations Research, 2012, 39(12): 2991-3011.

[49] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images[J]. Handbook of Systemic Autoimmune Diseases, 2009, 1(4).

[50] Miften M, Wiesmeyer M, Monthofer S, et al. Implementation of FFT convolution and multigrid superposition models in the FOCUS RTP system[J]. Physics in Medicine & Biology, 2000, 45(4):817-833.

[51] Carrion M, Arroyo J M. A computationally efficient mixed-integer linear formulation for the thermal unit commitment problem[J]. IEEE Transactions on Power Systems, 2006, 21(3):1371-1378.

[52] Moreland K, Angel E. The FFT on a GPU[C]//Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware. 2003: 112-119.

[53] Lavin A, Gray S. Fast algorithms for convolutional neural networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 4013-4021.

[54] Cho K, Merrienboer B V, Gulcehre C, et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation[J]. Computer Science, 2014.

此页不缺内容

致 谢

岁月如白驹过隙，三年的求学生涯已经接近尾声，而这段时间给我的成长和进步让我满怀感激和不舍。书不尽言，唯有感谢。

首先要感谢我的导师，冯晓兵研究员。冯老师和善慈祥，严谨睿智，每周都坚持给我们开讨论班，对于毕业的各个时间节点都亲自关注并投入指导。在科研提出了较高的要求，用关心和鼓励，帮助我进步。今后也会谨记老师的教导，精进不休。

感谢崔慧敏老师，严谨的科研作风和认真的科研态度是我学习的榜样。崔老师从不因为工作上的失误责备我们，而是报以微笑，以温和的态度面对我们。崔老师在生活中的态度也值得我们学习，乐观积极并能享受自己的生活，以快乐年轻的心态面对每一天。

感谢赵家程老师，赵老师平易近人，和学生打成一片，在工作和学习中也能愿意解答我们极其幼稚的问题。本次论文从选题到完成，每一步都是在导师的悉心指导下完成的，倾注了赵老师的心血。除了毕业论文，在科研项目上赵老师也帮助我很多，教会我很多做事的方法，大到工作的方法论，小到工程上的程序问题，赵老师都能耐心地指导，让我深感自己是个幸运的研究生。

感谢李坚松师兄，李师兄在工程上的能力超出了绝大多数同龄人，在毕业论文的研究过程中，李师兄的专业素质和修改意见，使我的论文提升了一个档次。李师兄广博的学识和丰富的工程经验，是我崇拜的对象也是学习的对象。

感谢刘一伊，林川清，孙乾，徐文栋，吴佳琦，杨雨昕，杨胜杰，周文辉，林苏颖，王卫染等等各个计算所的好朋友，是你们让我的研究生三年更加精彩和丰富，想到一起并肩作战找工作，也想到一起举办活动，参加聚会，我会觉得三年的时光太短。

感谢父母和兄长，你们的支持和鼓励是我不断前行的动力，谢谢一直以来对于我的关心照顾和包容。

最后我想感谢冯雪晴，你就像奇迹一般地出现在我的生命中，让我觉得前路一片光明，也让我对未来充满了憧憬。正如你说的那样，我期待着和你一起温柔而又坚定地走下去。

作者简历及攻读学位期间发表的学术论文与研究成果

姓名：杨永新 性别：男 出生日期：1996 年 2 月 15 日 籍贯：山西
2014 年 9 月——2018 年 6 月，在天津大学电气自动化与信息工程学院获得学士学位。
2018 年 9 月——2021 年 6 月，在中国科学院大学计算技术研究所获得硕士学位。

【攻读硕士学位期间发表的论文】

[1] Jiansong Li,Guangli Li,Xueying Wang,Yongxin Yang,Xiaobing Feng.Pinpointing the Memory Behaviors of DNN Training [C]/2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). IEEE, 2021.

【攻读硕士学位期间参加的科研项目】

- [1] 2019/9-2020/1 面向深度学习的 FPGA 编译器
- [2] 2020/8-2020/12 面向异构融合数据流加速器的编程模型及编译器优化

【攻读硕士学位期间的获奖情况】

- [1] 2019-2020 年度计算所三好学生
- [2] 2019-2020 年度计算所优秀学生干部